

Abstract

Many tractor owners and irrigation service providers still manage their daily work, customer information, expenses, and payments using notebooks, leading to misplaced records, calculation errors, delayed payments, and difficulties in tracking historical data. **AgriWorks Manager – Smart Farm Work & Irrigation Management System** replaces this manual method with a **deployed web application** designed for **tractor owners, irrigation service providers, and agricultural contractors**. The system provides **farmer management, agricultural work recording** for Ploughing, Rotavator, Cultivation, Harvesting, Irrigation, Land Leveling, and Other Work, along with field location and remarks, **automatic area-based and time-based billing, bill generation** with Unpaid/Partial/Paid tracking, **payment recording** with automatic pending amount recalculation, **expense management, dashboards, and reports** with filters and CSV export. It includes **secure login with 24-hour token expiry, password reset, problem reporting with admin handling, login history monitoring, and English/Hindi/Marathi language support with Light/Dark Mode** on all devices. The frontend in **React.js** is to be hosted on **Vercel**, the **Django REST backend** on **Render**, and **MySQL** on **Aiven**. All **55 test cases passed with zero failures**, confirming a reliable digital solution that **saves time, improves accuracy, and simplifies agricultural service business management**.

CHAPTER 1: INTRODUCTION

1.1 Background

Tractor owners and irrigation service providers in rural areas use notebooks and paper to note down their daily work, customer data, expenses, and payments. This manual processing is tedious and error-prone. Mistakes in calculations, misplaced papers, delayed payments, and inability to quickly locate prior work are just a few examples of how such a system is less than ideal. More and more farmers are realizing the importance of going paperless and are turning to digital solutions to help them run their businesses more effectively and efficiently.

AgriWorks Manager is a web-based application designed to help agricultural workers by collecting all their data in one place and organizing it so that it is easy to navigate and understand. Users will be able to store their farmer's data records, keep track of all their work, such as ploughing, rotavator, cultivation, harvesting, and land leveling, and manage invoicing. Expenses and payments can also be tracked and organized and viewed whenever required. The application enables users to invoice their work depending on the area for land preparation or in hours and minutes for irrigation works. The user can also enter the job location and remarks concerning the work done to identify the work. This saves time and helps users manage their business in a more organized and systematic manner.

1.2 Objectives

The main objective of this project is to design a web application that will enable tractor owners, irrigation service providers, and agricultural contractors to operate their daily businesses online and replace the use of hardcopy records.

The project will also ensure **automated billing**, record-keeping of payments received, and monitoring of expenses and reports of all kinds to help with the ease of managing all kinds of businesses.

It is expected to provide solutions for **small agricultural service providers**.

Objectives

- To develop a simple and user-friendly web-based application for tractor owners and irrigation service providers.
- To replace the **manual record management system** with a digital system.
- To manage farmers' details and daily work records effectively.
- To automate **bill generation** and ensure the recording of payments made and received.
- To monitor income and expenditure.
- To accommodate various agricultural works, including ploughing, rotavator, cultivating, harvesting, irrigation, land leveling, and other works.
- To ensure automatic calculation of job cost depending on the area covered or irrigation period and applicable rates.
- To record work location and remarks.
- To provide a **complaint or problem reporting module**.
- To provide an administrator dashboard to monitor the entire system, including users' activities, complaint reports, and active user logins.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The objective of this project is to digitize the manual work of agricultural service businesses, thus allowing users to record agricultural service work using a web-based application instead of multiple notebooks. This system will enable the users to achieve greater efficiency, minimize manual work resulting from paperwork, save time with ease of use, and reduce calculation errors while keeping accurate records of their work.

The system is designed to provide an intuitive interface for users with no **technical expertise**, enabling them to record work, access farmer details, and maintain financial records. It ensures that farmers have a clear account of their income, expenses, and payments with ease.

1.3.2 Scope of the Project

- **Agricultural work activity manager** for tractor owners, irrigation service providers, and agricultural contractors.
- Registration & Login (Email, Username, and Mobile Number) for secure access, and Password Reset & Change facility.
- Farmer & Agricultural Work Record, Billing, Payment, Expense, and Report record manager.
- **Work Types:** Ploughing, Rotavator, Cultivation, Harvesting, Irrigation, Land Leveling, and Other work.
- Area-based billing (Area and Rate per Acre) and Time-based billing (Hours, Minutes, and Rate per Hour) features automatically calculate work charges to reduce manual errors.
- Users can record Field Location and Remarks for completed work. Additionally, **Work Description** for Other work.
- Bill record from unbilled work with Unpaid / Partial / Paid status tracking and Payment record with Pending Amount calculation.
- Expenses tracker for Diesel, Maintenance, Driver Wages, and Other.
- Dashboard with business totals and Reports with filters and CSV Export for organized access to the business records.
- **Contact & Report Problem** feature with screenshot upload for users to submit reports and track the status of their complaint with the following statuses: Pending, In Progress, and Resolved.
- Admin Dashboard for viewing registered users, reported problems, and successful login activity records.
- Multilingual Support: English, Hindi, and Marathi with Light / Dark mode.
- Designed for small and medium-sized agricultural service businesses.
- Responsive web application with accessibility through browsers on computers, tablets, and mobile phones.

1.3.3 Applicability

This application can be used by Tractor Owners, Irrigation Service Providers, and Agricultural Contractors to manage their day-to-day work activities more efficiently and keep track of their business records digitally using an intuitive interface. Their manual work is digitized, eliminating the need for multiple notebooks for record keeping.

The application is suitable for managing various agricultural work activities such as ploughing, rotavator, cultivation, harvesting, irrigation, land leveling, and other agricultural work. The system also offers **area-based billing** (Area and Rate per Acre) and Time-based billing (Hours, Minutes, and Rate per Hour) features to help users keep track of their unbilled work. Thus, the user can easily prepare bills from unbilled work while eliminating calculation errors that are common with manual work recording.

Users can keep track of their completed work by recording the Field Location and Remarks. This feature also enables users to search and update completed work. The description of completed work can also be recorded using the Work Description for Other work feature.

The application helps users to monitor their business expenses (Diesel, Maintenance, Driver Wages, and Other) while keeping track of their business payments (Bill record from unbilled work with Unpaid / Partial / Paid status tracking, and Payment record with Pending Amount calculation). Using the application's Dashboard, the user can monitor business analytics and record their business reports with filters and CSV Export for organized business data access. The **Contact & Report Problem** feature empowers users to submit their problems and monitor the status of their complaints (Pending, In Progress, and Resolved).

The application's Admin Dashboard helps track the registered users, reported problems, and successful login activities. The application is multilingual (English, Hindi, and Marathi), and it comes with a Light / Dark mode. It is designed to serve small and medium-sized agricultural service businesses and is a responsive web application with accessibility through browsers on computers, tablets, and mobile phones.

This application can be used in diverse agricultural service settings to help minimize the use of manual work records, thus eliminating the need for multiple notebooks. The

system will enable agricultural service businesses to offer better services, keep track of their income, expenses, and payments with accuracy and ease. The digitization of the manual work records will empower the agricultural service business owners to access their work records easily since they will be stored digitally. As a result, the owners will have a clear account of their business activities, enabling them to make informed business decisions.

1.4 Achievements

The AgriWorks Manager system has been successfully designed, implemented, and deployed. The frontend is hosted on Vercel, the backend on Render, and the database on Aiven MySQL. Achievements are:

- Management of farmers, agricultural work, bills, payments, and expenses implemented.
- **Automatic calculation of area-based charges and time-based irrigation charges** is working.
- Bill generation from unbilled work with Unpaid / Partial / Paid status implemented.
- Payment tracking with pending amount calculation and overpayment rejection implemented.
- Expense tracking, Dashboard totals, and Reports with filters and CSV export implemented.
- User authentication with password reset, Contact & Report Problem with status tracking, and Admin Dashboard with user monitoring and login history implemented.
- **Responsive, multilingual (English/Hindi/Marathi) interface with Light/Dark mode** deployed and accessible on computers, tablets, and mobile devices.

1.5 Organization of Report

Chapter 1 presents the **background, objectives, purpose, scope, and achievements** of the project.

Chapter 2 presents the survey of technology, including the study and analysis of 15 existing research works and systems related to agricultural service management.

Chapter 3 describes the **problem definition, requirements, planning with Gantt and PERT charts, and conceptual models**.

Chapter 4 explains module division and system interfaces.

Chapter 5 covers implementation approaches and the working of the project.

Chapter 6 presents test cases and test reports.

Chapter 7 gives limitations, future enhancements, and conclusion, followed by References.

CHAPTER 2 SURVEY OF TECHNOLOGIES

1. Farm Management Information Systems Review

Title: Farm management information systems: Current situation and future perspectives

Author: S. Fountas, G. Carli, C.G. Sørensen, Z. Tsiropoulos, C. Cavalaris, A. Vatsanidou, B. Liakos, M. Canavari, J. Wiebensohn, B. Tisserye

Publish Date/Year: 2015

Journal/Conference: Computers and Electronics in Agriculture

Publisher: Elsevier

Summary: Reviews academic and commercial FMIS for open-field crops, analysing architectures, adoption, and precision-agriculture solutions across 141 commercial packages. Finds academic systems stronger in standards and interoperability while commercial ones focus on recordkeeping, finance, and machinery. Core FMIS reference for AgriWorks records, billing, and machinery scope.

2. Future FMIS Concept

Title: Conceptual model of a future farm management information system

Author: C.G. Sørensen, S. Fountas, E. Nash, L. Pesonen, D. Bochtis, S.M. Pedersen, B. Basso, S.B. Blackmore

Publish Date/Year: 2010

Journal/Conference: Computers and Electronics in Agriculture

Publisher: Elsevier

Summary: Develops a conceptual model defining FMIS boundaries, stakeholders, data flows, and functional requirements for next-generation systems. Foundational requirements reference for AgriWorks farmer records, work recording, and billing design.

3. Future Internet Farm Systems

Title: Farm management systems and the Future Internet era

Author: A. Kaloxylos, R. Eigenmann, F. Teye, Z. Politopoulou, S. Wolfert, C. Shrank, M. Dillinger, I. Lampropoulou, E. Antoniou, L. Pesonen, H. Nicole, F. Thomas, N. Alonistioti, G. Kormentzas

Publish Date/Year: 2012

Journal/Conference: Computers and Electronics in Agriculture

Publisher: Elsevier

Summary: Proposes Internet-connected FMIS services with IoT integration and a service-marketplace architecture instead of isolated desktop software. Justifies AgriWorks cloud-based design and interoperability.

4. FMIS Reference Architecture

Title: Reference architecture design for farm management information systems: a multi-case study approach

Author: J. Tummers, A. Kassahun, B. Tekinerdogan

Publish Date/Year: 2021 (online-first 2020)

Journal/Conference: Precision Agriculture

Publisher: Springer Nature

Summary: Proposes an FMIS-specific reference architecture with guidelines and a method to derive application architectures, validated via multi-case studies. Blueprint for AgriWorks farmer, work, billing, and reporting modules.

5. Digital Recordkeeping Review

Title: From pen and paper to digital precision: a comprehensive review of on-farm recordkeeping

Author: Md.S. Basir, D.R. Buckmaster, A. Raturi, Y. Zhang

Publish Date/Year: 2024

Journal/Conference: Precision Agriculture

Publisher: Springer Nature

Summary: Reviews paper versus digital recordkeeping across 29 tools and 31 apps, assessing interoperability and accuracy. Finds most tools paid with weak export support. Evidence base for AgriWorks digital records with CSV export.

6. Farm Cost Accounting (FarmBO)

Title: Introducing Activity-Based Costing in Farm Management: The Design of the FarmBO System

Author: Giacomo Carli, Maurizio Canavari, Alessandro Grandi

Publish Date/Year: 2014

Journal/Conference: International Journal of Agricultural and Environmental Information Systems

Publisher: IGI Global

Summary: Proposes an FMIS model with Activity-Based Costing and a working FarmBO prototype giving crop balance sheets and what-if simulations, validated on two farms. Cost-management reference behind AgriWorks billing and expense reports.

7. Machinery Decision Support (MAChoice)

Title: MAChoice: a Decision Support System for agricultural machinery management

Author: M. Cunha, S.G. Gonçalves

Publish Date/Year: 2019

Journal/Conference: Open Agriculture

Publisher: De Gruyter

Summary: Presents a web-based DSS estimating machinery costs and break-even points under different use rates, with cost and emission charts, demonstrated on grape harvesting. Machinery-cost reference behind AgriWorks work-rate decisions.

8. IoT Irrigation with Machine Learning

Title: An IoT based smart irrigation management system using Machine learning and open source technologies

Author: Amarendra Goap, Deepak Sharma, A. K. Shukla, C. Rama Krishna

Publish Date/Year: 2018

Journal/Conference: Computers and Electronics in Agriculture

Publisher: Elsevier

Summary: Pilot-deployed open-source system sensing soil and climate plus weather forecasts, predicting soil moisture with machine learning for irrigation scheduling with web dashboard. Closest irrigation match for AgriWorks sensing-to-dashboard flow.

9. SWAMP Water Platform

Title: Smart Water Management Platform: IoT-Based Precision Irrigation for Agriculture

Author: Carlos Kamienski, Juha-Pekka Soininen, Markus Taumberger, Ramide Dantas, Attilio Toscano, Tullio Salmon Cinotti, Rodrigo Filev Maia, André Torre Neto

Publish Date/Year: 2019

Journal/Conference: Sensors

Publisher: MDPI

Summary: Describes the SWAMP IoT irrigation platform tested on four pilots in Brazil and Europe with FIWARE components, stressing replicability and scalability. Reference architecture for cloud irrigation platforms.

10. Wireless Sensor Networks Survey

Title: Wireless sensor networks for agriculture: The state-of-the-art in practice and future challenges

Author: Tamoghna Ojha, Sudip Misra, Narendra Singh Raghuwanshi

Publish Date/Year: 2015

Journal/Conference: Computers and Electronics in Agriculture

Publisher: Elsevier

Summary: Reviews WSN devices, sensors, and communication techniques with Indian and global case studies, listing deployment prospects, problems, and future directions. Sensor-network foundation for future AgriWorks irrigation links.

11. Ubiquitous Sensor Platform

Title: Developing Ubiquitous Sensor Network Platform Using Internet of Things: Application in Precision Agriculture

Author: Francisco Javier Ferrández-Pastor, Juan Manuel García-Chamizo, Mario Nieto-Hidalgo, Jerónimo Mora-Pascual, José Mora-Martínez

Publish Date/Year: 2016

Journal/Conference: Sensors

Publisher: MDPI

Summary: Builds a low-cost heterogeneous sensor network with edge computing, validated in greenhouse hydroponics controlling irrigation, climate, and nutrients. Low-cost automation model for AgriWorks irrigation.

12. Big Data Smart Farming Review

Title: Big Data in Smart Farming – A review

Author: Sjaak Wolfert, Lan Ge, Cor Verdouw, Marc-Jeroen Bogaardt

Publish Date/Year: 2017

Journal/Conference: Agricultural Systems

Publisher: Elsevier

Summary: Framework for Big Data across the food chain with predictive insights and real-time decisions, warning on power shifts and stressing data standards and governance. Frames AgriWorks IoT-cloud-report cycle.

13. Agriculture 5.0 Data Review

Title: From Smart Farming towards Agriculture 5.0: A Review on Crop Data Management

Author: Verónica Saiz-Rubio, Francisco Rovira-Más

Publish Date/Year: 2020

Journal/Conference: Agronomy

Publisher: MDPI

Summary: Reviews farm systems from sensor data acquisition to variable-rate application, arguing robotics and AI data use avoids resource misuse. Decision-support reference for AgriWorks data tools.

14. Smart Farming Perspective

Title: Smart farming is key to developing sustainable agriculture

Author: Achim Walter, Robert Finger, Robert Huber, Nina Buchmann

Publish Date/Year: 2017

Journal/Conference: Proceedings of the National Academy of Sciences

Publisher: National Academy of Sciences

Summary: Argues sensor networks, phenotyping, and precision interventions raise productivity sustainably, linking field sensing to farm management and policy. High-level justification for AgriWorks.

15. Agriculture 4.0 Perspective

Title: Agriculture 4.0: Broadening Responsible Innovation in an Era of Smart Farming

Author: David Christian Rose, Jason Chilvers

Publish Date/Year: 2018

Journal/Conference: Frontiers in Sustainable Food Systems

Publisher: Frontiers Media SA

Summary: Argues Agriculture 4.0 tech needs farmer inclusion and testing frameworks so farming is not undesirably re-scripted. Guides farmer-inclusive AgriWorks design.

Comparative Analysis

The surveyed research covers farm management information systems, recordkeeping, cost accounting, machinery decision support, IoT irrigation, sensor networks, big data, and responsible innovation. Fountas et al., Sørensen et al., Kaloxyllos et al., and Tummers et al. establish FMIS architectures, requirements, and cloud-based futures; Basir et al. confirms the shift from pen-and-paper to digital records with export needs. Carli et al. and Cunha et al. address farm costing and machinery decisions but for crop production, not service billing. Goap et al., Kamienski et al., Ojha et al., and Ferrández-Pastor et al. advance sensing and irrigation control without customer or financial management. Wolfert et al., Saiz-Rubio et al., Walter et al., and Rose et al. frame data-driven and responsible smart farming without implementing service workflows.

AgriWorks Manager – Smart Farm Work & Irrigation Management System addresses this gap through a deployed web application for tractor owners, irrigation service providers, and agricultural contractors. It combines farmer management, work recording, billing, payment tracking, expense management, and reporting with area-based billing for applicable services and time-based billing for irrigation. The system reduces manual record keeping and calculation errors and simplifies financial and business management.

Technology Selection

The following technology stack has been selected for developing AgriWorks Manager:

React.js: Interactive and user-friendly frontend.

Bootstrap 5: Responsive layouts for different devices.

Python (Django): Backend development and business logic.

Django REST Framework: REST API development and frontend-backend communication.

React Router: Page navigation and protected routes.

Axios: Frontend-backend API communication.

Vite: React development and build tool.

MySQL: Relational database for users, farmers, work, billing, payments, expenses, problem reports, and login history.

Vercel (frontend), **Render** (backend), **Aiven** (MySQL hosting): Deployment and hosting.

The selected technologies provide a suitable combination of usability, reliability, maintainability, and responsive performance for the proposed agricultural service management system.

CHAPTER 3 REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

Many tractor owners and irrigation service providers maintain daily work, farmer details, bills, payments, and expenses in notebooks. This manual system causes misplaced records, calculation mistakes in area-based and time-based charges, delayed payments, and difficulty in tracking past work and pending amounts. There is a need for a simple web-based system that maintains farmer records, work records with location and remarks, automatic billing, payment tracking, expense management, and reports in one place, with an Admin facility to monitor users, problem reports, and login activity.

3.2 Requirements Specification

Functional Requirements:

- User registration and login with email, username, or registered mobile number, logout, password reset and password change.
- Farmer management with add, view, edit, delete, and search (name, mobile, village, address).
- Work management for Ploughing, Rotavator, Cultivation, Harvesting, Irrigation, Land Leveling, and Other, with work date, field location, remark, and work description for Other work.
- Automatic calculation of area-based charges (area x rate per acre) and irrigation charges ((hours + minutes/60) x hourly rate).
- Bill generation from unbilled work with Unpaid / Partial / Paid status.
- Payment tracking with pending amount calculation; overpayment is rejected.
- Expense management under Diesel, Maintenance, Driver Wages, and Other.
- Dashboard with business totals and Reports (work, billing, payments, pending payments, expenses, business performance) with filters and CSV export.

- Contact & Report Problem with screenshot upload and status tracking as Pending, In Progress, Resolved.
- Admin Dashboard for monitoring registered users, problem reports, and successful login history.

Non-Functional Requirements:

- Responsive web access on computer, tablet, and mobile through compatible browsers.
- Multilingual interface (English / Hindi / Marathi) with Light / Dark mode.
- Secure token-based access with 24-hour expiry and single active session per account.
- Simple interface suitable for users without advanced technical knowledge.

3.3 Software and Hardware Requirements

Table 3.1: Software Requirements

Sr. No.	Component	Specification
1	Frontend Framework	React.js 19
2	Routing	React Router 7
3	API Communication	Axios
4	UI Framework	Bootstrap 5.3
5	Build Tool	Vite 8, Node.js ≥ 20 , npm
6	Backend Language	Python
7	Backend Framework	Django 6.1.1
8	API Framework	Django REST Framework 3.18.1
9	Server / Libraries	Gunicorn, MySQL Client
10	Database	MySQL 8.x (Aiven Hosted)
11	Browser	Chrome / Edge / Firefox (Latest)
12	Hosting	Vercel (Frontend), Render (Backend), Aiven (Database)

Table 3.2: Hardware Requirements

Sr. No.	Component	Specification
1	Processor	Dual-core or above
2	RAM	4 GB minimum (8 GB recommended)
3	Storage	10 GB free space; hosted database for production
4	Devices	Computer / Tablet / Mobile with Internet Connection

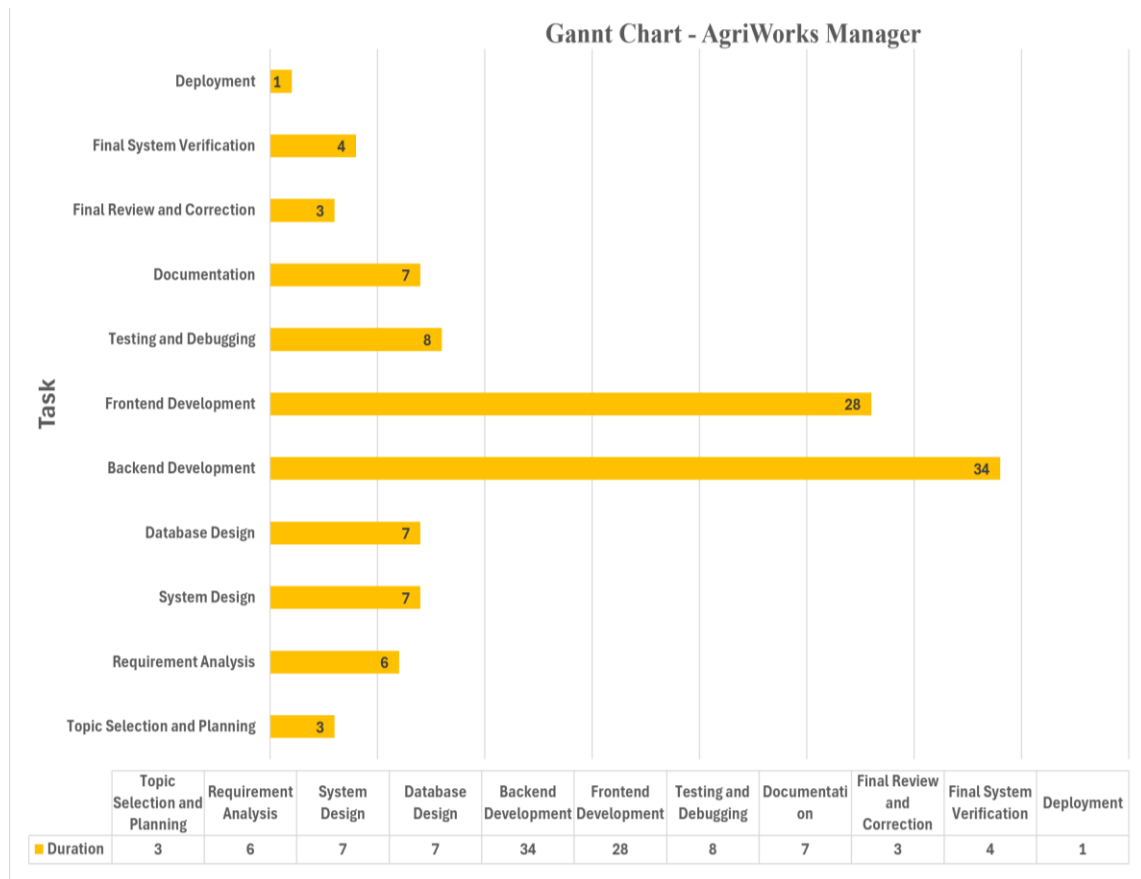
3.4 Planning and Scheduling

Planning and scheduling are important parts of software project development. They divide the project into activities and provide a structured approach for completion. **AgriWorks Manager** includes activities such as requirement analysis, system design, database design, frontend and backend development, testing, and documentation.

3.4.1 Gantt Chart

The Gantt Chart shows project activities, duration, and schedule. It helps track progress, identify delays, and complete tasks on time.

S.No.	Task	Assigned To	Start Date	End Date	Duration
1	Topic Selection and Planning	Kaushik & Samiksha	15 June 2026	17 June 2026	3 Days
2	Requirement Analysis	Kaushik & Samiksha	18 June 2026	23 June 2026	6 Days
3	System Design	Kaushik & Samiksha	24 June 2026	30 June 2026	7 Days
4	Database Design	Kaushik & Samiksha	01 July 2026	07 July 2026	7 Days
5	Backend Development	Kaushik & Samiksha	08 July 2026	10 August 2026	34 Days
6	Frontend Development	Kaushik & Samiksha	11 August 2026	07 September 2026	28 Days
7	Testing and Debugging	Kaushik & Samiksha	08 September 2026	15 September 2026	8 Days
8	Documentation	Kaushik & Samiksha	16 September 2026	22 September 2026	7 Days
9	Final Review and Correction	Kaushik & Samiksha	23 September 2026	25 September 2026	3 Days
10	Final System Verification	Kaushik & Samiksha	26 September 2026	29 September 2026	4 Days
11	Deployment	Kaushik & Samiksha	30 September 2026	30 September 2026	1 Days



The Gantt Chart represents the project activities and their planned duration using horizontal bars.

3.4.2 PERT Chart

PERT stands for **Program Evaluation and Review Technique**. It represents the sequence and dependencies of project activities using a network diagram. It helps identify the critical path, estimate project completion time, and improve project planning. The three main time estimates are:

- **Optimistic Time (O):** Minimum expected time required.
- **Most Likely Time (M):** Normal expected time required.
- **Pessimistic Time (P):** Maximum expected time required.

Expected Time:- $TE = (O + 4M + P) / 6$,

Standard Deviation:- $SD = (P - O) / 6$

Variance:- $Variance = (SD)^2$

Proposed PERT Activities:

Task (T)	O	M	P	Expected Time (TE)	S.D.	Variance
T1 Topic Selection and Planning	2	3	4	3.00	0.33	0.11
T2 Requirement Analysis	4	6	8	6.00	0.67	0.44
T3 System Design	5	7	9	7.00	0.67	0.44
T4 Database Design	5	7	9	7.00	0.67	0.44
T5 Backend Development	29	34	39	34.00	1.67	2.78
T6 Frontend Development	23	28	33	28.00	1.67	2.78
T7 Testing and Debugging	6	8	10	8.00	0.67	0.44
T8 Documentation	6	7	8	7.00	0.33	0.11
T9 Final Review and Correction	2	3	4	3.00	0.33	0.11
T10 Final System Verification	3	4	5	4.00	0.33	0.11
T11 Deployment	1	1	2	1.17	0.17	0.03

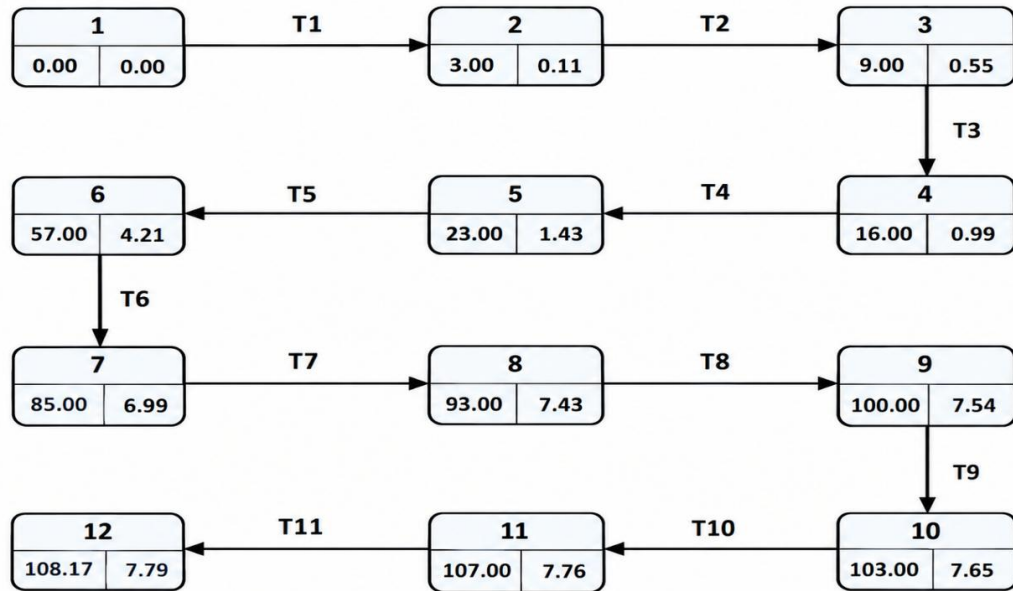
Node No.	Activity Completed	Expected Time (TE)	Cumulative Time	Variance	Cumulative Variance
1	Start	0.00	0.00	0.00	0.00
2	T1	3.00	3.00	0.11	0.11
3	T2	6.00	9.00	0.44	0.55
4	T3	7.00	16.00	0.44	0.99
5	T4	7.00	23.00	0.44	1.43
6	T5	34.00	57.00	2.78	4.21
7	T6	28.00	85.00	2.78	6.99
8	T7	8.00	93.00	0.44	7.43
9	T8	7.00	100.00	0.11	7.54
10	T9	3.00	103.00	0.11	7.65
11	T10	4.00	107.00	0.11	7.76
12	T11	1.17	108.17	0.03	7.79

Formula used:

- **Cumulative Time** = Previous Cumulative Time + Current TE
- **Cumulative Variance** = Previous Cumulative Variance + Current Variance

Final Project Expected Time = 108.17 Days

Final Cumulative Variance = 7.79

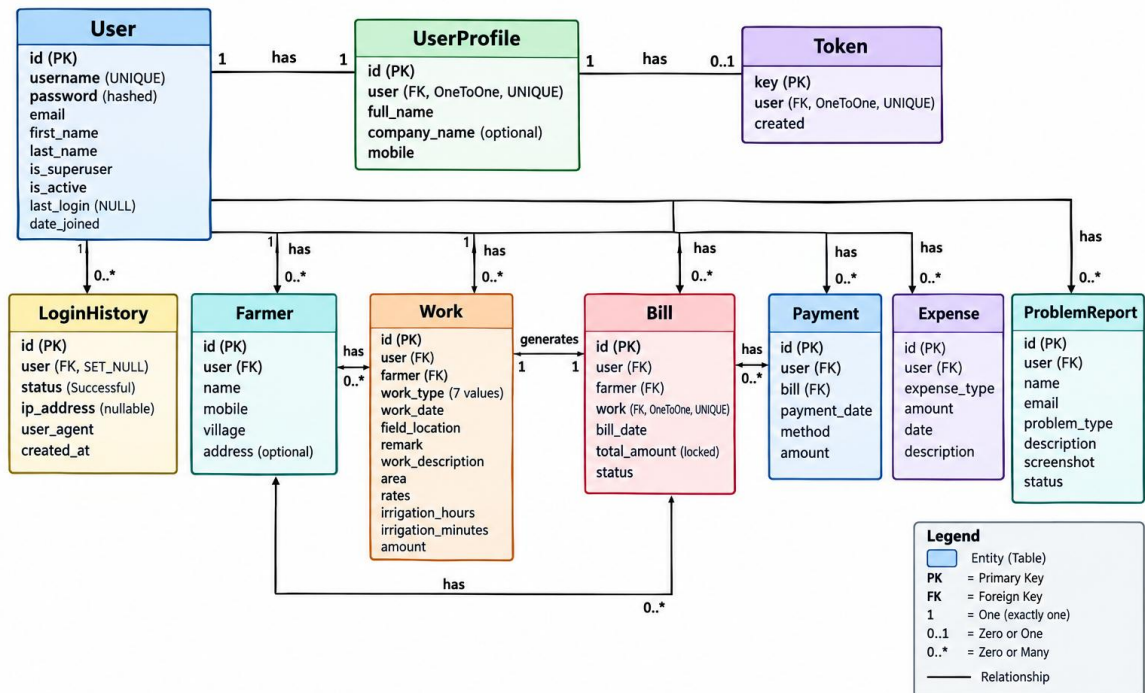


3.5 Conceptual Models

Conceptual models represent the structure and working of the proposed system. They help understand **entities, data relationships, data flow, system structure, user activities, and interactions**. The models prepared for the proposed system include:

1. Entity Relationship Diagram (ER Diagram)
2. Data Dictionary
3. Data Flow Diagram – Level 0 & Data Flow Diagram – Level 1
4. Class Diagram
5. Activity Diagram
6. Use Case Diagram
7. System Flowchart

3.5.1 Entity Relationship Diagram



3.5.2 Data Dictionary

A Data Dictionary describes the main data fields used in the system.

User Table

Field Name	Data Type	Description
id	INT AUTO INCREMENT	Primary Key, unique user identifier
username	VARCHAR(150)	Unique username used for login
password	VARCHAR(128)	Hashed password used for user authentication
email	VARCHAR(254)	Registered email address of the user
first_name	VARCHAR(150)	User's full name entered during registration; stored in Django's first_name field
last_name	VARCHAR(150)	User's last name entered during registration and profile update
is_superuser	TINYINT(1)	Identifies whether the user has administrator access
is_staff	TINYINT(1)	Django staff flag; it is not used by the application's admin authorization logic
is_active	TINYINT(1)	Indicates whether the user account is active
last_login	DATETIME(6) NULL	Date and time of the user's most recent login
date_joined	DATETIME(6)	Date and time when the user registered

Token Table

Field Name	Data Type	Description
key	VARCHAR(40)	Primary Key, unique authentication token
user	INT	One-to-One Foreign Key referencing the User table; each user can have one authentication token
created	DATETIME(6)	Date and time when the token was created

UserProfile Table

Field Name	Data Type	Description
id	BIGINT AUTO INCREMENT	Primary Key, cannot be NULL, unique profile identifier
user	INT	One-to-One Foreign Key, UNIQUE, referencing the User table and identifying the profile owner
full_name	VARCHAR(150)	Registered full name of the user
company_name	VARCHAR(150) NOT NULL	Business or company name; optional in the application, stored as an empty value when not provided
mobile	VARCHAR(10)	User's mobile number
created_at	DATETIME(6)	Date and time when the profile was created
updated_at	DATETIME(6)	Date and time when the profile was last updated

Login History Table

Field Name	Data Type	Description
id	BIGINT AUTO INCREMENT	Primary Key, cannot be NULL, unique login record identifier
user	INT	Foreign Key referencing the User table and identifying the user who logged in
status	VARCHAR(20)	Login status; currently stores Successful for successful login records
created_at	DATETIME(6)	Date and time of the successful login
ip_address	GenericIPAddressField NULL	IP address from which the user logged in
user_agent	VARCHAR(255)	Browser and client information recorded during login

Farmer Table

Field Name	Data Type	Description
id	BIGINT AUTO INCREMENT	Primary Key, cannot be NULL, unique farmer identifier
user	INT	Foreign Key referencing the User table and identifying the owner of the farmer record
name	VARCHAR(100)	Name of the farmer
mobile	VARCHAR(10)	Farmer's mobile number
village	VARCHAR(100)	Village where the farmer is located
address	LONGTEXT	Farmer's address; optional in the application and stored as an empty value when not provided
created_at	DATETIME(6)	Date and time when the farmer record was created
updated_at	DATETIME(6)	Date and time when the farmer record was last updated

Expense Table

Field Name	Data Type	Description
id	BIGINT AUTO INCREMENT	Primary Key, cannot be NULL, unique expense identifier
user	BIGINT Foreign Key	References the User table and identifies the expense owner
expense_type	VARCHAR(20)	Type of expense such as Diesel, Maintenance, Driver Wages, or Other
amount	DECIMAL(10,2)	Amount spent on the expense
date	DATE	Date on which the expense occurred
description	LONGTEXT NOT NULL	Additional details about the expense; optional in the interface and stored as an empty value when not provided
created_at	DATETIME(6)	Date and time when the expense record was created
updated_at	DATETIME(6)	Date and time when the expense record was last updated

Work Table

Field Name	Data Type	Description
id	BIGINT AUTO INCREMENT	Primary Key, cannot be NULL, unique work record identifier
user	INT	Foreign Key referencing the User table and identifying the record owner
farmer	BIGINT	Foreign Key referencing the Farmer table and identifying the farmer for whom the work was performed
work_type	VARCHAR(20)	Type of agricultural work: Ploughing, Rotavator, Cultivation, Harvesting, Irrigation, Land Leveling, or Other
field_location	VARCHAR(200)	Location where the agricultural work was performed; required when creating or updating a work record
remark	LONGTEXT	Additional notes related to the work; optional
work_description	LONGTEXT	Description of the work; required when the work type is Other
irrigation_hours	INT UNSIGNED	Number of hours used for irrigation
irrigation_minutes	INT UNSIGNED	Number of additional minutes used for irrigation
hourly_rate	DECIMAL(10,2) NULL	Rate charged per hour for irrigation work
rate_per_acre	DECIMAL(10,2) NULL	Rate charged per acre for area-based agricultural work
work_date	DATE	Date on which the agricultural work was performed
area	DECIMAL(10,2) NULL	Area of land covered by the work, measured in acres
amount	DECIMAL(10,2)	Total amount charged for the agricultural work; calculated by the system except for Other Work, where it is entered manually
created_at	DATETIME(6)	Date and time when the work record was created
updated_at	DATETIME(6)	Date and time when the work record was last updated

Bill Table

Field Name	Data Type	Description
id	BIGINT AUTO INCREMENT	Primary Key, cannot be NULL, unique bill identifier
user	BIGINT Foreign Key	References the User table and identifies the bill owner
farmer	BIGINT Foreign Key	References the Farmer table and identifies the billed farmer
work	BIGINT Foreign Key, UNIQUE	References the Work table; one work record can have one bill
bill_date	DATE	Date on which the bill was generated
total_amount	DECIMAL(10,2)	Fixed total amount of the generated bill. It should not be changed after bill generation
status	VARCHAR(10), Default = Unpaid	Payment status of the bill: Unpaid, Partial, or Paid
created_at	DATETIME(6)	Date and time when the bill was created
updated_at	DATETIME(6)	Date and time when the bill was last updated

Payment Table

Field Name	Data Type	Description
id	BIGINT AUTO INCREMENT	Primary Key, cannot be NULL, unique payment identifier
user	BIGINT Foreign Key	References the User table and identifies who recorded the payment
bill	BIGINT Foreign Key	References the Bill table and identifies the bill against which the payment is recorded
payment_date	DATE	Date on which the payment was received
method	VARCHAR(20)	Payment method such as Cash, UPI, Bank Transfer, Cheque, or Other
amount	DECIMAL(10,2)	Amount received from the farmer
created_at	DATETIME(6)	Date and time when the payment record was created
updated_at	DATETIME(6)	Date and time when the payment record was last updated

Problem Report Table

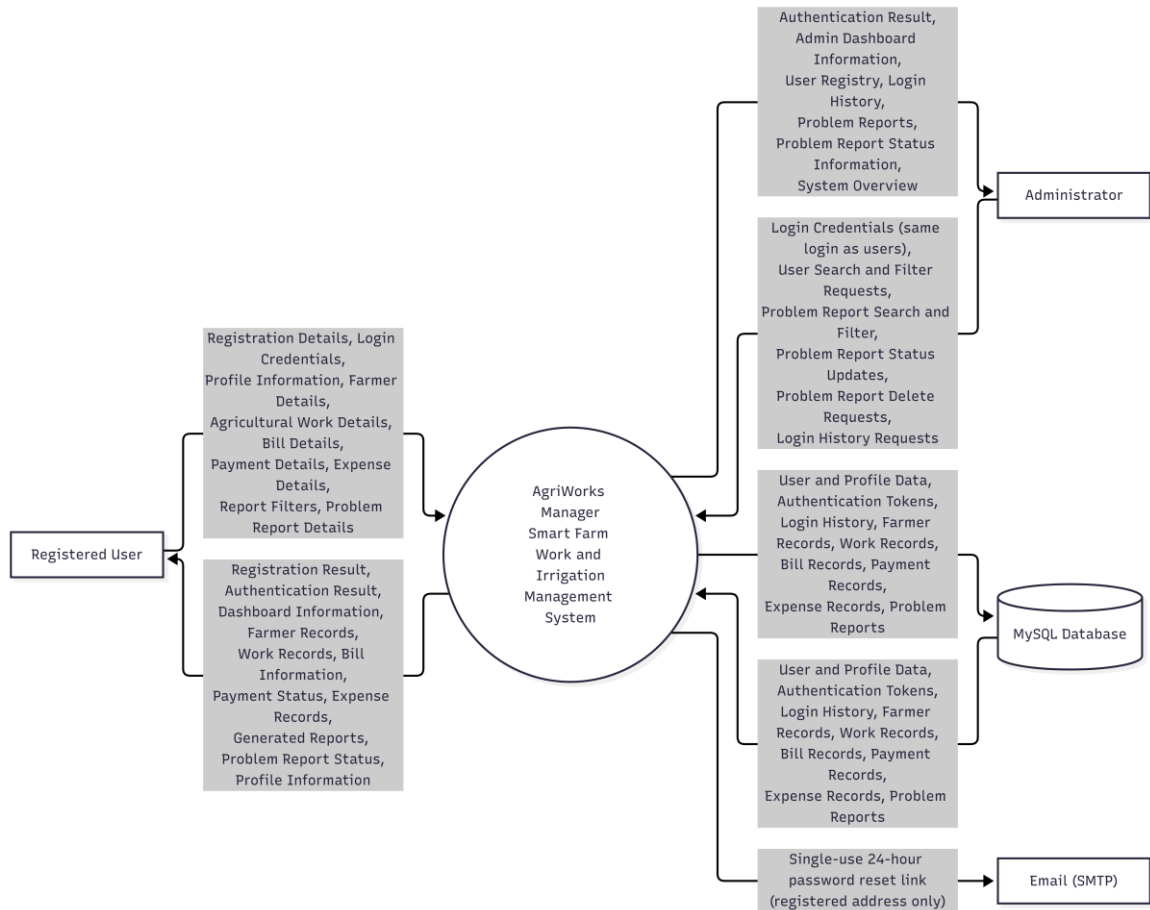
Field Name	Data Type	Description
id	BIGINT AUTO INCREMENT	Primary Key, cannot be NULL, unique problem report identifier
user	BIGINT Foreign Key	References the User table and identifies the user who submitted the report
name	VARCHAR(150)	Name of the person reporting the problem
email	VARCHAR(254)	Email address of the reporter
problem_type	VARCHAR(30)	Category of the reported problem such as Login Problem, Work Record Problem, Billing Problem, Payment Problem, Report Problem, Dashboard Problem, Language Problem, or Other
description	LONGTEXT	Detailed description of the reported problem
screenshot	VARCHAR(100) NULL	Optional screenshot provided as evidence of the problem; supports image files up to 2 MB
status	VARCHAR(20), Default = Pending	Current status of the report: Pending, In Progress, or Resolved
created_at	DATETIME(6)	Date and time when the problem report was submitted
updated_at	DATETIME(6)	Date and time when the problem report was last updated

Database Relationships

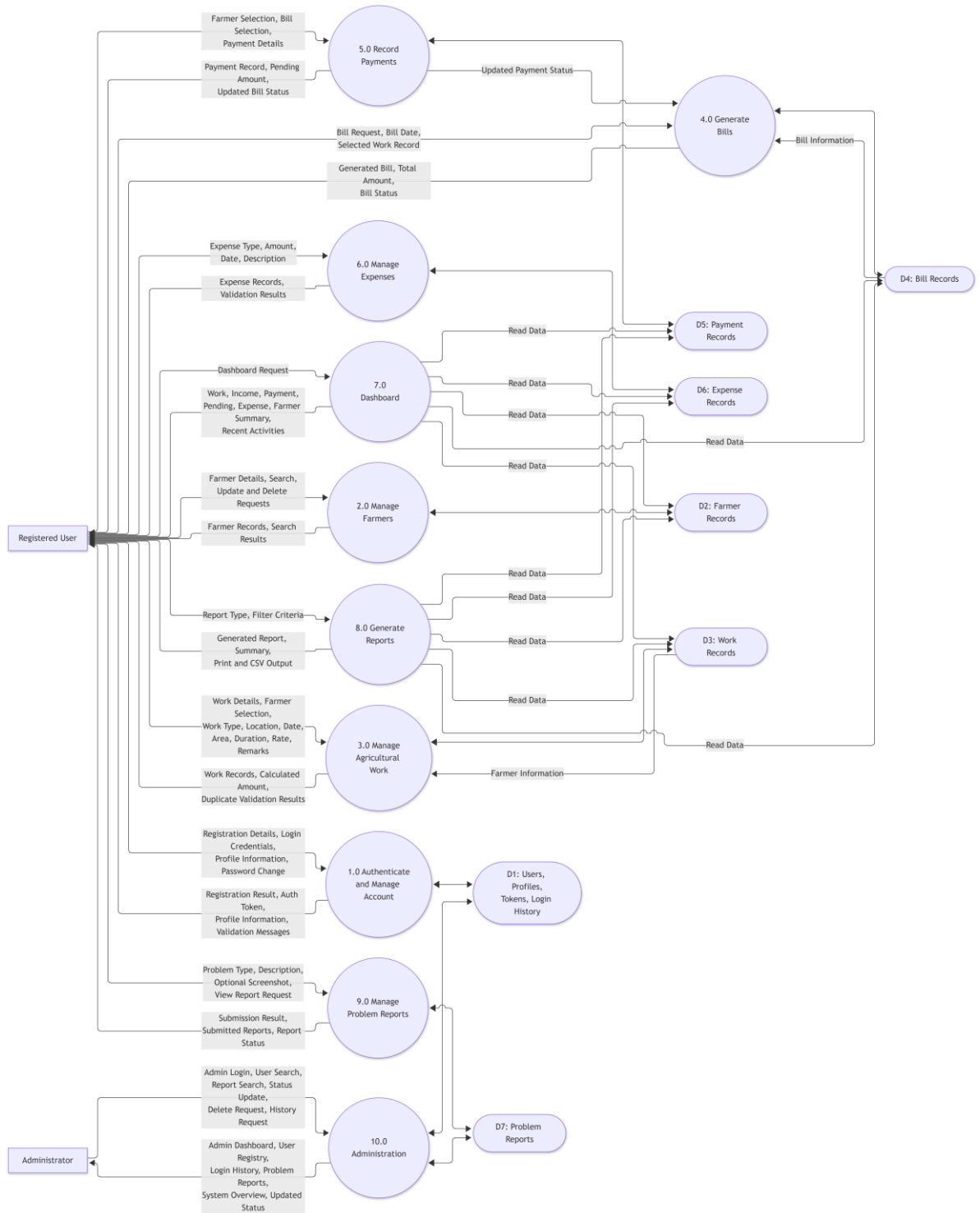
The main relationships between the tables are:

- **User → UserProfile: One user has one profile. (1:1)**
- **User → Token: One user has at most one authentication token. (1:1)**
- **User → Login History: One user can have multiple login history records. (1:M)**
- **User → Farmer: One user can manage multiple farmers. (1:M)**
- **User → Work: One user can create multiple work records. (1:M)**
- **Farmer → Work: One farmer can have multiple work records. (1:M)**
- **User → Bill: One user can have multiple bills. (1:M)**
- **Farmer → Bill: One farmer can have multiple bills. (1:M)**
- **Work → Bill: One work record can have one bill. (1:1)**
- **User → Payment: One user can record multiple payments. (1:M)**
- **Bill → Payment: One bill can have multiple payment records. (1:M)**
- **User → Expense: One user can record multiple expenses. (1:M)**
- **User → Problem Report: One user can submit multiple problem reports. (1:M)**

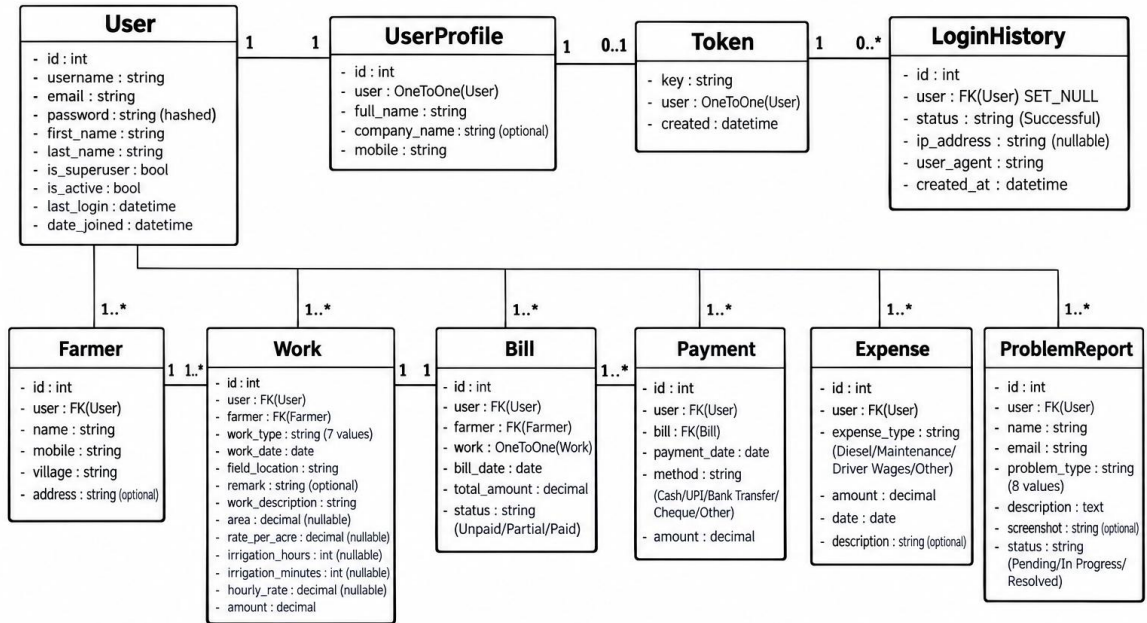
3.5.3 Data Flow Diagram – Level 0 & Data Flow Diagram – Level 1



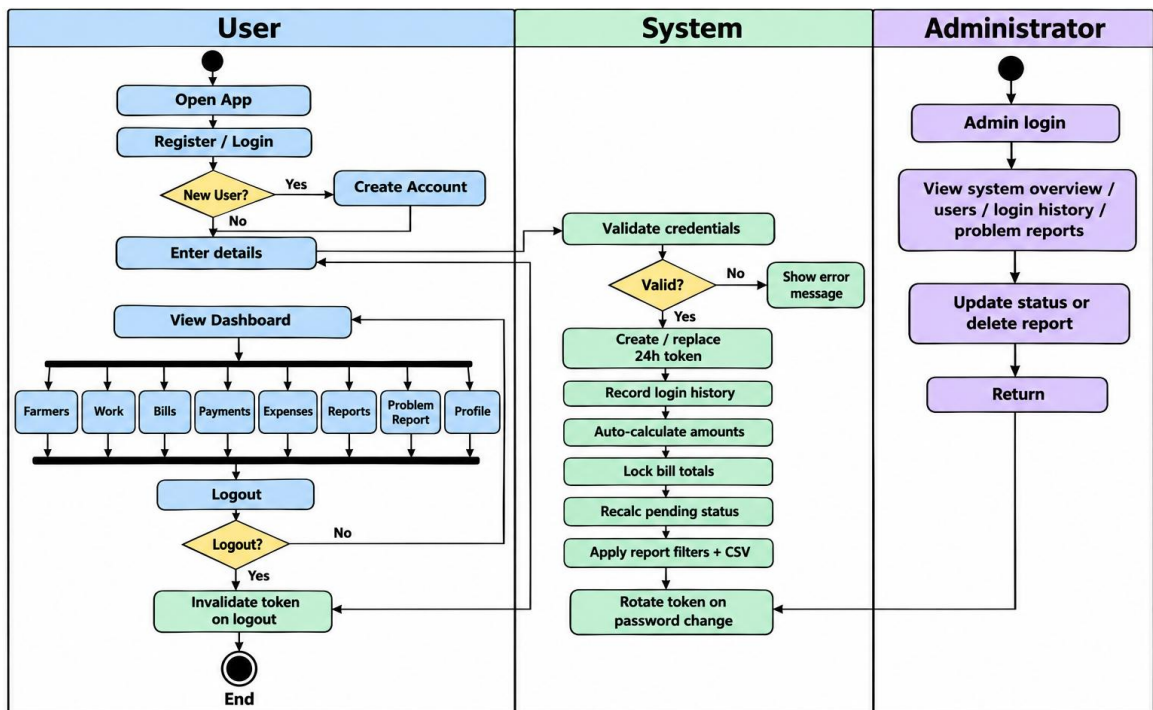
Data Flow Diagram – Level 1



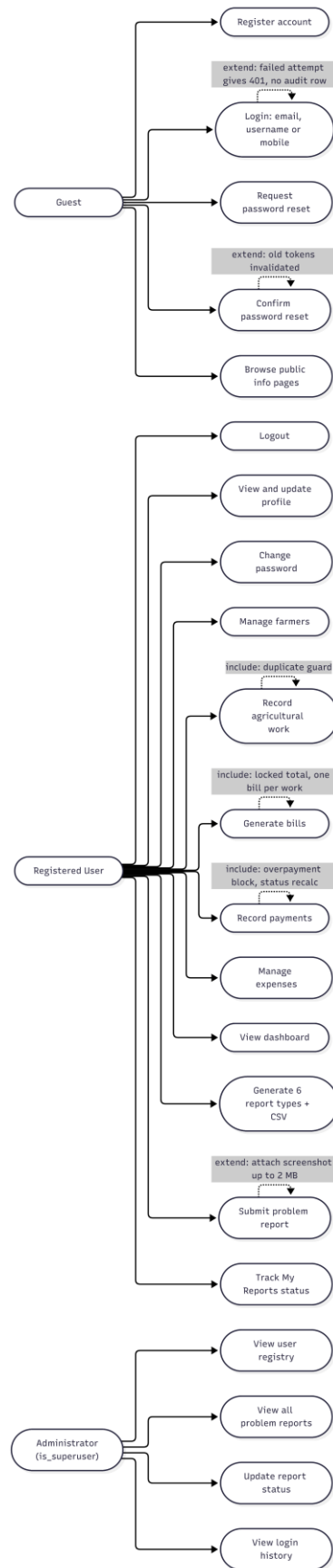
3.5.4 Class Diagram



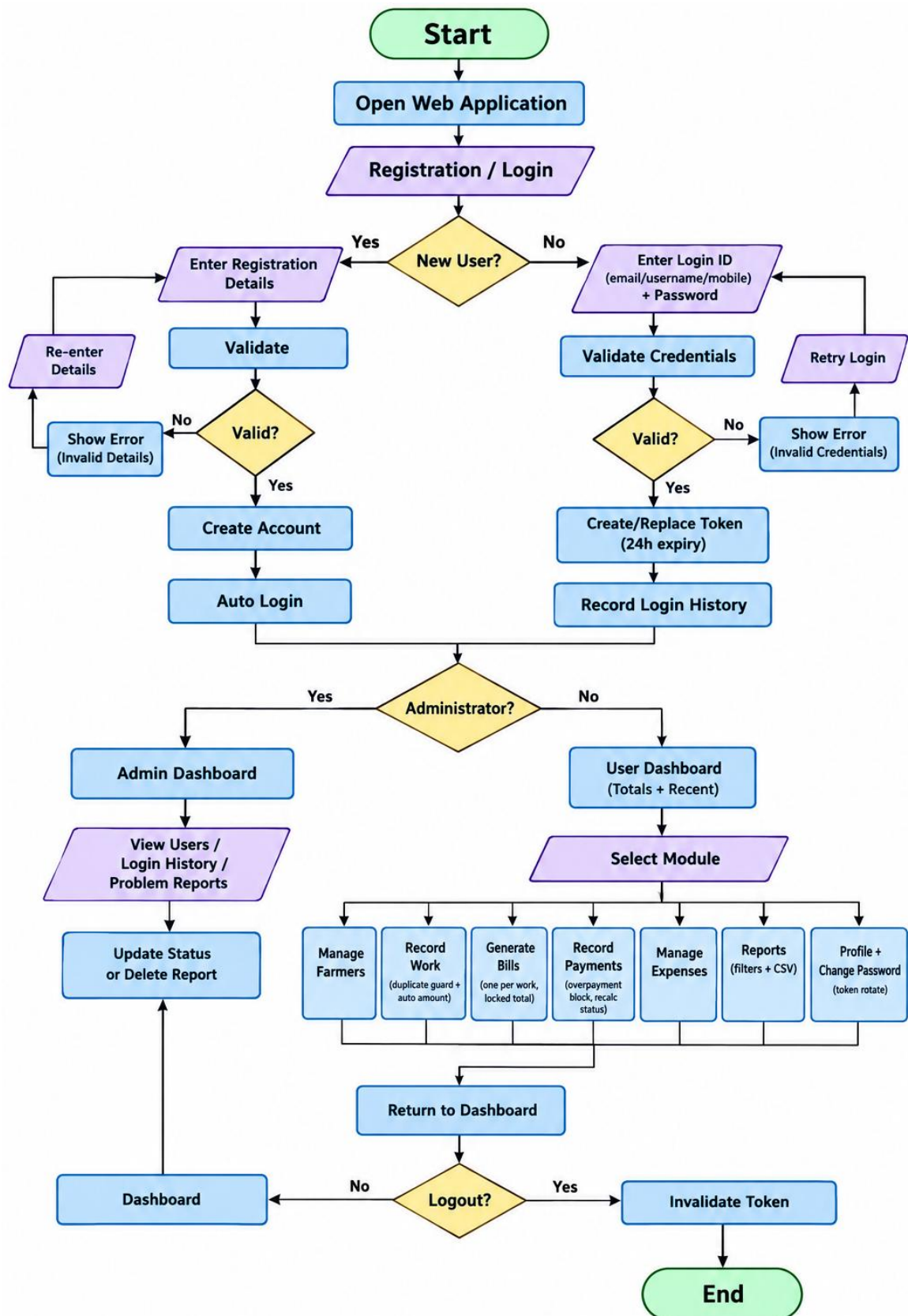
3.5.5 Activity Diagram



3.5.6 Use Case Diagram



3.5.7 System Flowchart



Chapter 3 Summary

This chapter described the requirements and analysis of **AgriWorks Manager** – Smart Farm Work & Irrigation Management System.

The chapter covered problem definition, requirements specification, software and hardware requirements, planning and scheduling with Gantt Chart and PERT Chart, and conceptual models including Entity Relationship Diagram, Data Dictionary, Data Flow Diagram, Class Diagram, Activity Diagram, and Use Case Diagram.

The requirements section defined the functional and non-functional requirements of the system, including user authentication, user profile management, farmer management, agricultural work management, billing, payment tracking, expense management, dashboard, report generation, language support, problem reporting, admin dashboard, and login history.

The agricultural work management requirements include support for Ploughing, Rotavator, Cultivation, Harvesting, Irrigation, Land Leveling, and Other work. The system supports area-based and time-based billing, while allowing users to record work location, remarks, work description, and manual total amount for Other work.

The conceptual models provide an understanding of the system's entities, data relationships, data flow, classes, activities, and user interactions. The Data Dictionary further describes the main database tables and their fields.

These requirements and conceptual models provide the foundation for the design, development, testing, and implementation of the proposed AgriWorks Manager system.

CHAPTER 4: SYSTEM DESIGN

4.1 Module Division

The **AgriWorks Manager system** is divided into functional modules to organize and manage different activities of the farm work and irrigation management process. Each module is designed to perform a specific set of operations, while all modules work together to provide an integrated and efficient system.

Sr. No.	Module Name	Description
1	Authentication & Account Management	Handles user registration, login, logout, password management, profile management, authentication tokens, and login history.
2	Farmer Management	Allows users to add, view, search, update, and manage farmer information.
3	Agricultural Work Management	Manages agricultural work records, including work type, farmer, work date, field details, irrigation details, area, rates, and amount.
4	Billing Management	Generates and manages bills for recorded agricultural work and maintains bill status.
5	Payment Management	Records and manages payments made against bills and updates the corresponding bill status.
6	Expense Management	Maintains farm and business expenses such as diesel, maintenance, driver wages, and other expenses.
7	Dashboard & Reports	Provides summarized information, recent records, business statistics, and reports based on existing system data.
8	Problem Report Management	Allows users to submit problems with descriptions and optional screenshots and track their report status.
9	Administration	Provides authorized administrators with user registry, login history, problem report management, and other administrative functions.

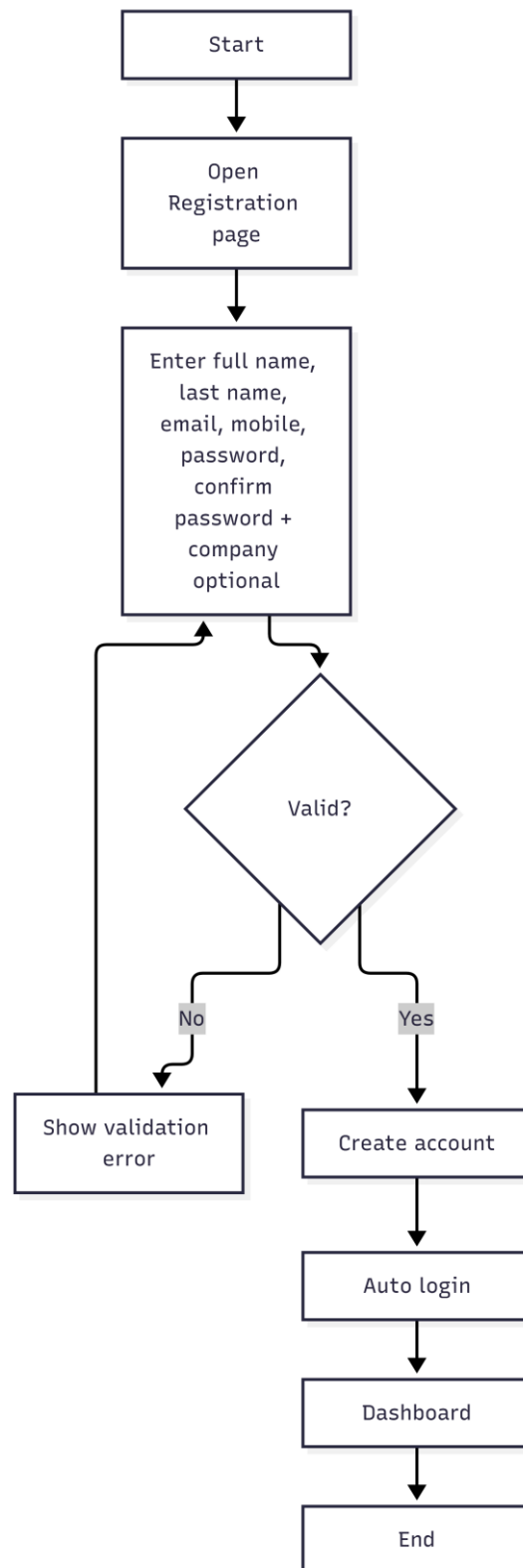
4.2 System Interface

4.2.1 User Registration

Algorithm:

1. Open the Registration page.
2. Enter full name, last name, email, mobile number, password, and confirm password.
3. Enter company name (optional).
4. Validate the entered details.
5. If invalid, display validation errors.
6. If valid, create the user account.
7. Automatically log in the user.
8. Redirect to the Dashboard.
9. End.

Flowchart:



Interface Diagram:

The screenshot shows the 'Create Account' page of the AgriWorks application. The header is green with the AgriWorks logo on the left and a language dropdown set to 'English' and a 'Login' button on the right. The main content area is light beige. The registration form is a white card with the title 'Create Account' and the subtitle 'Join AgriWorks — your digital register'. The form contains several input fields: 'Full Name *' and 'Last Name *' (both required), 'Company / Business Name (optional)' with a hint 'Leave blank if none', 'Email *' (required), and 'Mobile Number *' (required) which includes a '+91' country code selector and a prompt to 'Enter 10-digit mobile'. Below these are 'Password *' and 'Confirm Password *' (both required) fields, with a 'Show' button next to the password field. A green 'Create Account' button is at the bottom of the form. Below the button, there is a link: 'Already have an account? [Login](#)'. A password requirement note states: 'Min 8 characters with uppercase, lowercase, number and special character.'

Validation:

Full name, last name, email, mobile number, password, and confirm password are required. Mobile number must be a valid 10-digit number. Email must be valid and unique. Password must contain at least 8 characters with uppercase, lowercase, digit, and special character. **Confirm password must match the password.** Company name is optional.

Interface Description:

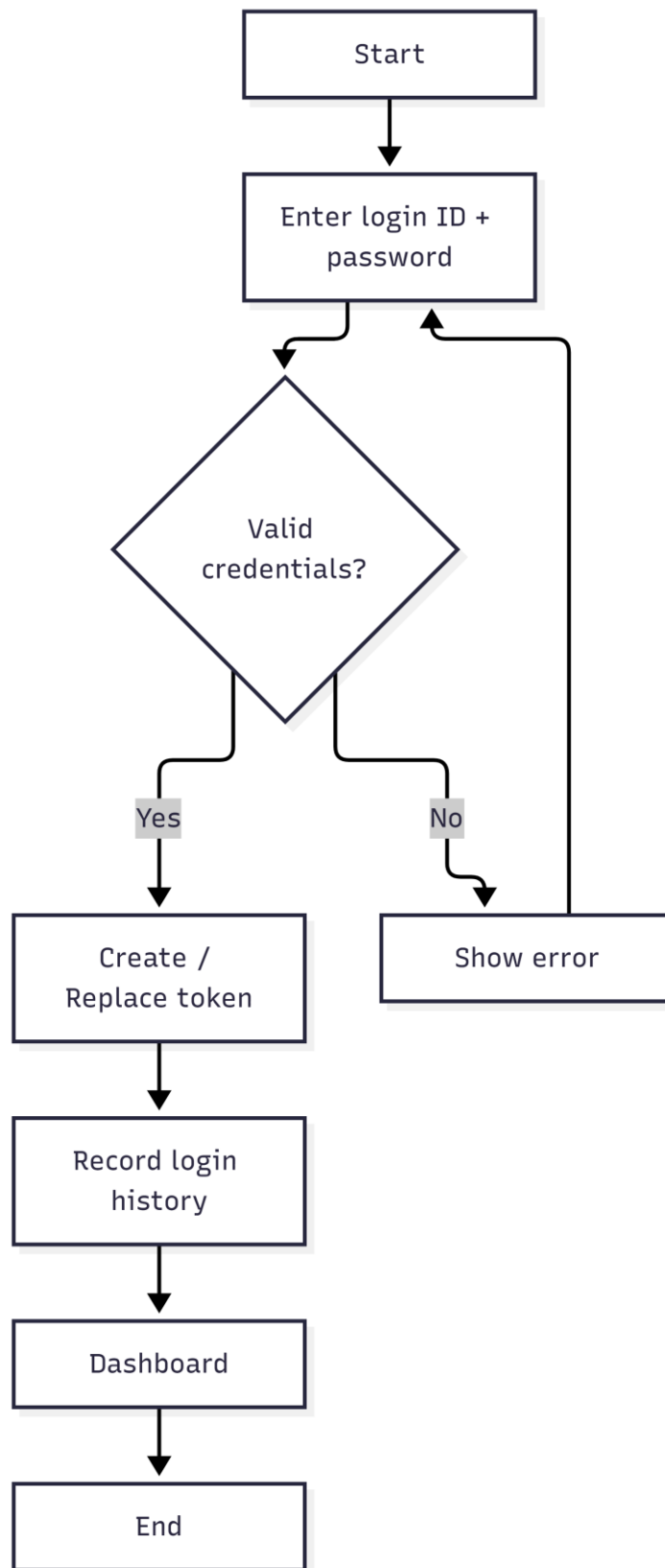
The Registration interface allows new users to create an account and automatically access the system after successful registration.

4.2.2 User Login

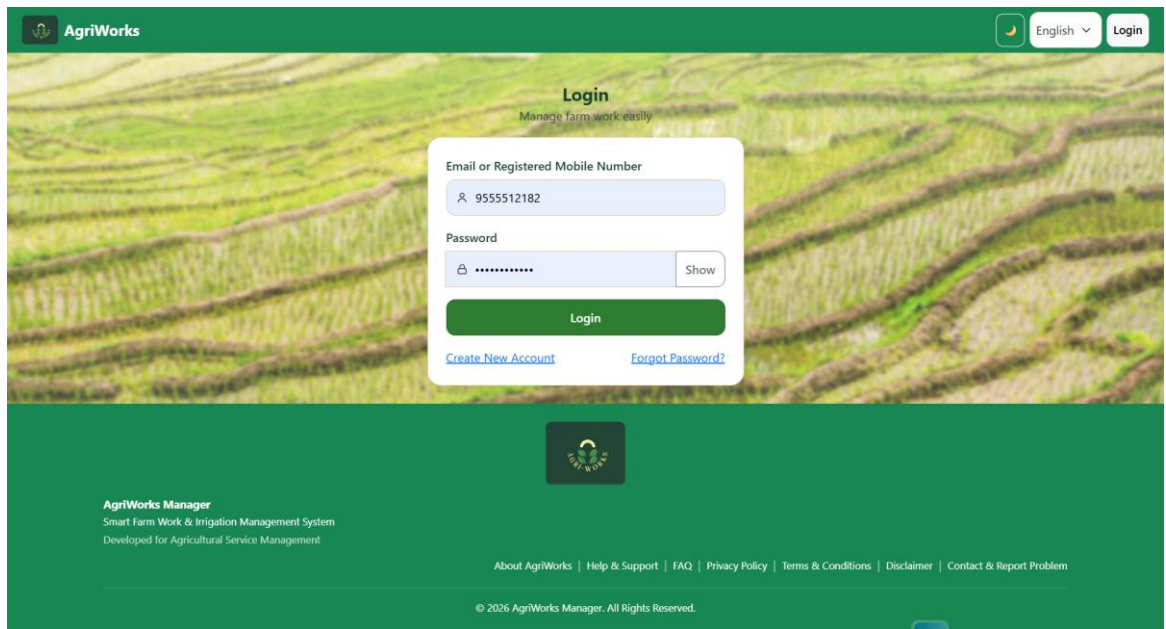
Algorithm:

1. Open the Login page.
2. Enter email, username, or registered mobile number.
3. Enter the password.
4. Validate the credentials.
5. If invalid, display an error message.
6. If valid, create or replace the authentication token.
7. Record the successful login in Login History.
8. Redirect the user to the appropriate dashboard.
9. End.

Flowchart:



Interface Diagram:



Validation:

Both login identifier and password are required. The authentication token is valid for 24 hours. Only one active session is maintained for a user.

Interface Description:

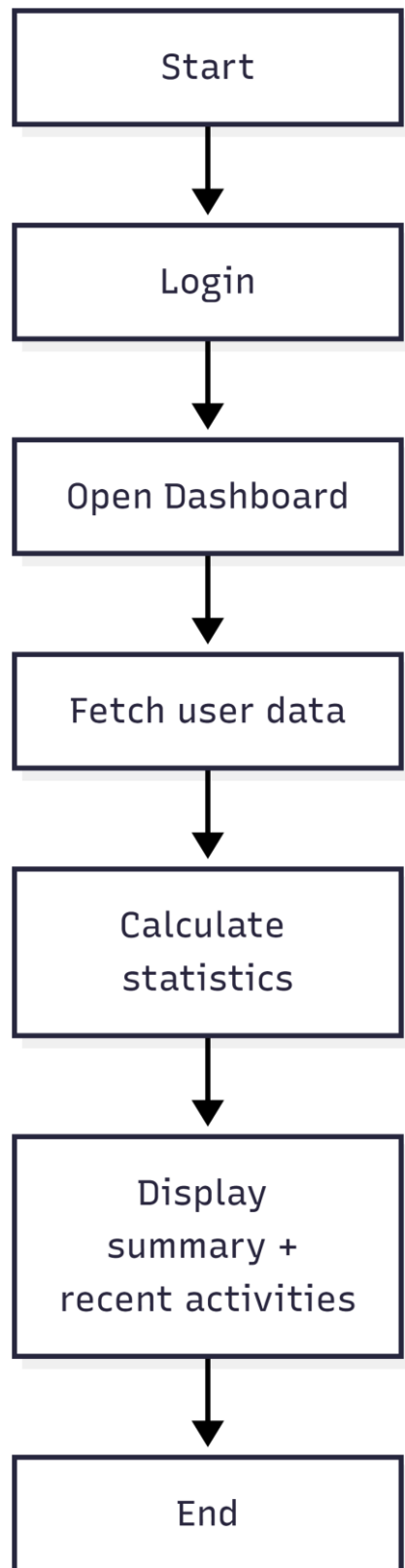
The Login interface allows users to securely access the system using email, username, or registered mobile number and password.

4.2.3 Dashboard

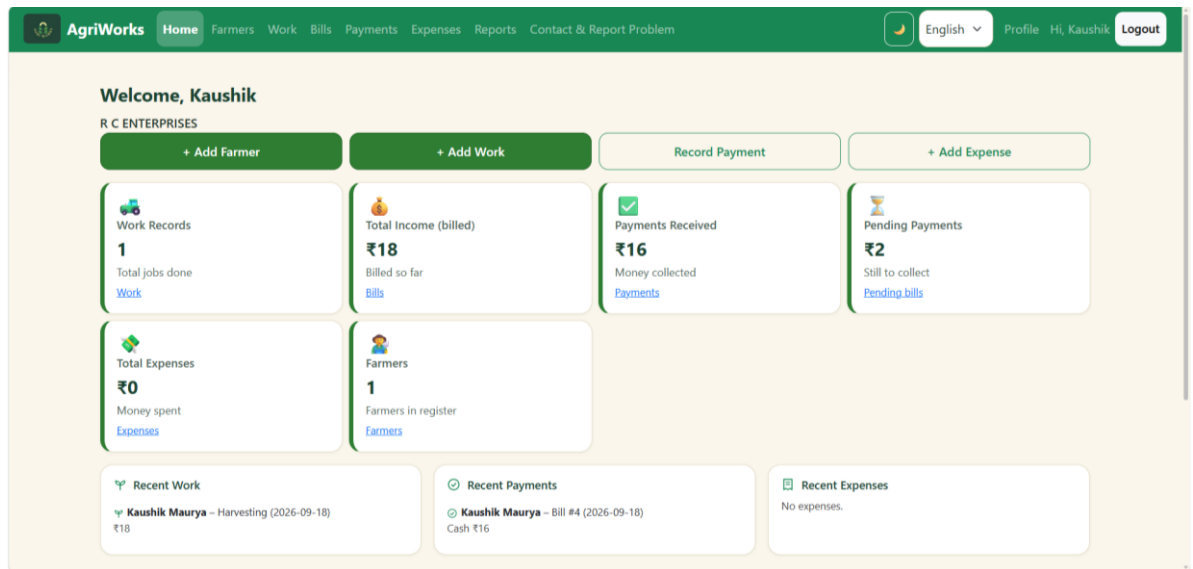
Algorithm:

1. Login to the system.
2. Open the Dashboard.
3. Fetch user-specific system data.
4. Calculate dashboard statistics.
5. Display summary cards and recent activities.
6. Provide quick-action options.
7. End.

Flowchart:



Interface Diagram:



Validation:

Only authenticated users can access the Dashboard. All displayed data is restricted to the logged-in user.

Interface Description:

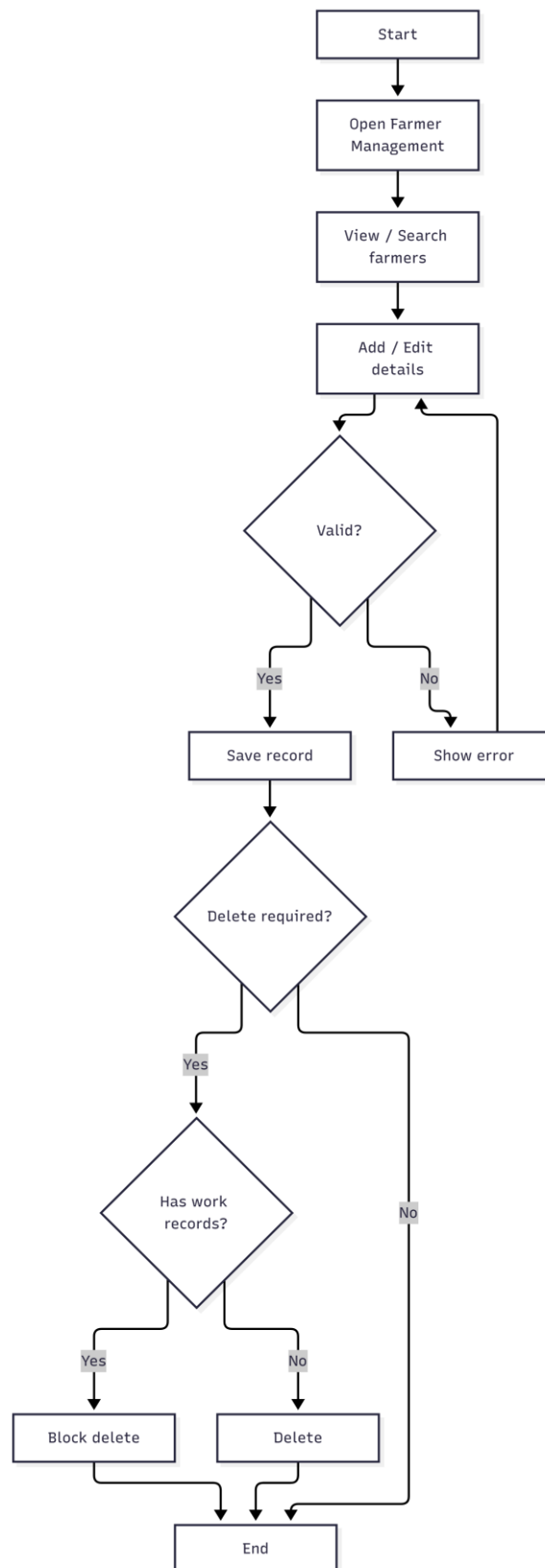
The Dashboard displays work records, billed income, received payments, pending payments, expenses, farmers, and recent activities.

4.2.4 Farmer Management

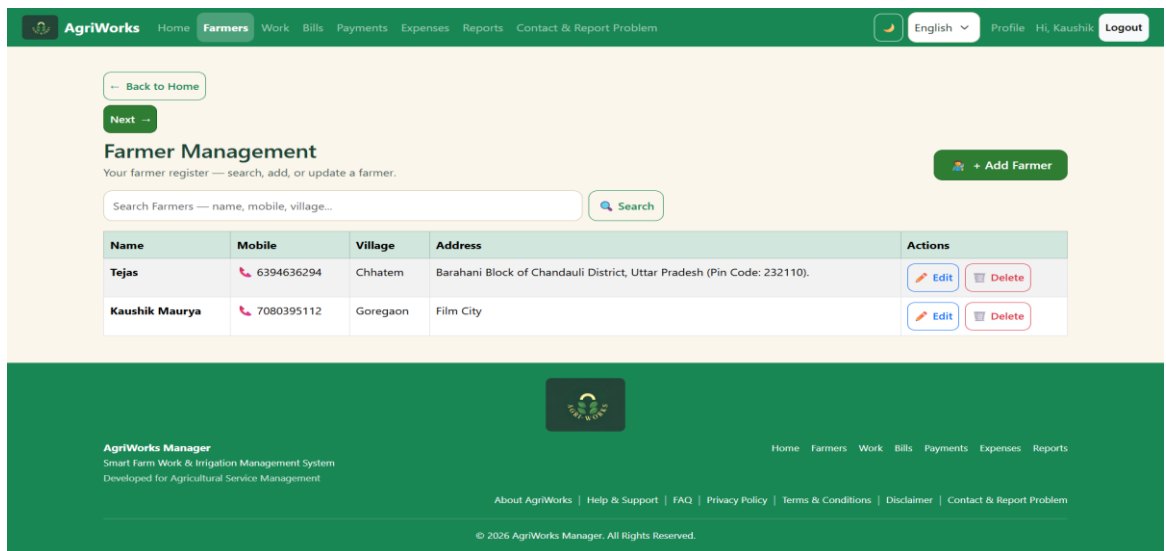
Algorithm:

1. Open Farmer Management.
2. View the farmer list.
3. Search farmer if required.
4. Add or edit farmer details if required.
5. Validate the entered information.
6. Save the farmer record.
7. Delete the record if permitted.
8. End.

Flowchart:



Interface Diagram:



Validation:

Name and village are required. Mobile number must be a valid 10-digit number. Address is optional. A farmer with existing work records cannot be deleted.

Interface Description:

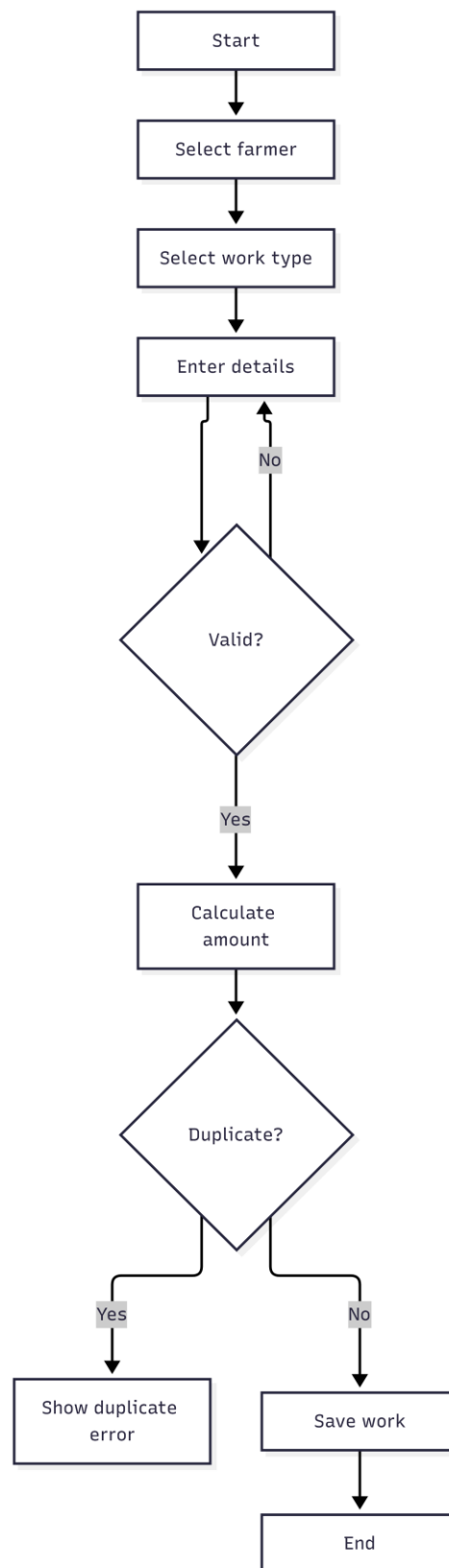
The Farmer Management interface allows users to create, view, search, update, and delete their farmer records.

4.2.5 Agricultural Work Management

Algorithm:

1. Open Work Management.
2. Select a farmer.
3. Select the work type.
4. Enter the required work details.
5. Validate the entered information.
6. Calculate the amount according to the selected work type.
7. Check for duplicate work records.
8. Save the work record.
9. View or delete the record if required.
10. End.

Flowchart:



Interface Diagram:

The screenshot shows the 'Agricultural Work' form in the AgriWorks Manager. The form is titled 'Agricultural Work' with a subtitle 'Record farm work for a farmer — it becomes the bill later.' It includes a search bar, filters for 'All farmers' and 'All work types', and a '+ Add Work' button. The form is divided into three sections: 1. Farmer (with a dropdown for 'Tejas (Chhatem)'), 2. Work Type (with buttons for 'Ploughing', 'Rotavator', 'Cultivation', 'Harvesting', 'Irrigation', 'Other', and 'Land Leveling'), and 3. Date & Place (with fields for 'Work date *' (24-09-2026) and 'Field / Location *' (North Field)). There is also a 'Note (optional)' field.

The screenshot shows the 'Agricultural Work' table in the AgriWorks Manager. The table has columns for Farmer, Work Type, Date, Area, Amount, and Actions. It contains two rows of data: one for 'Tejas' with 'Rotavator' work type, and one for 'Kaushik Maurya' with 'Harvesting' work type. Each row has 'Locked' and 'Delete' buttons in the Actions column.

Farmer	Work Type	Date	Area	Amount	Actions
Tejas	Rotavator	2026-09-24	1.20	₹960	<button>Locked</button> <button>Delete</button>
Kaushik Maurya	Harvesting	2026-09-18	1.50	₹18	<button>Locked</button> <button>Delete</button>

Validation:

Work date cannot be in the future. Location is required. Required fields depend on the selected work type. Duplicate work for the same farmer, work type, and date is not allowed. Work records cannot be edited after creation.

Interface Description:

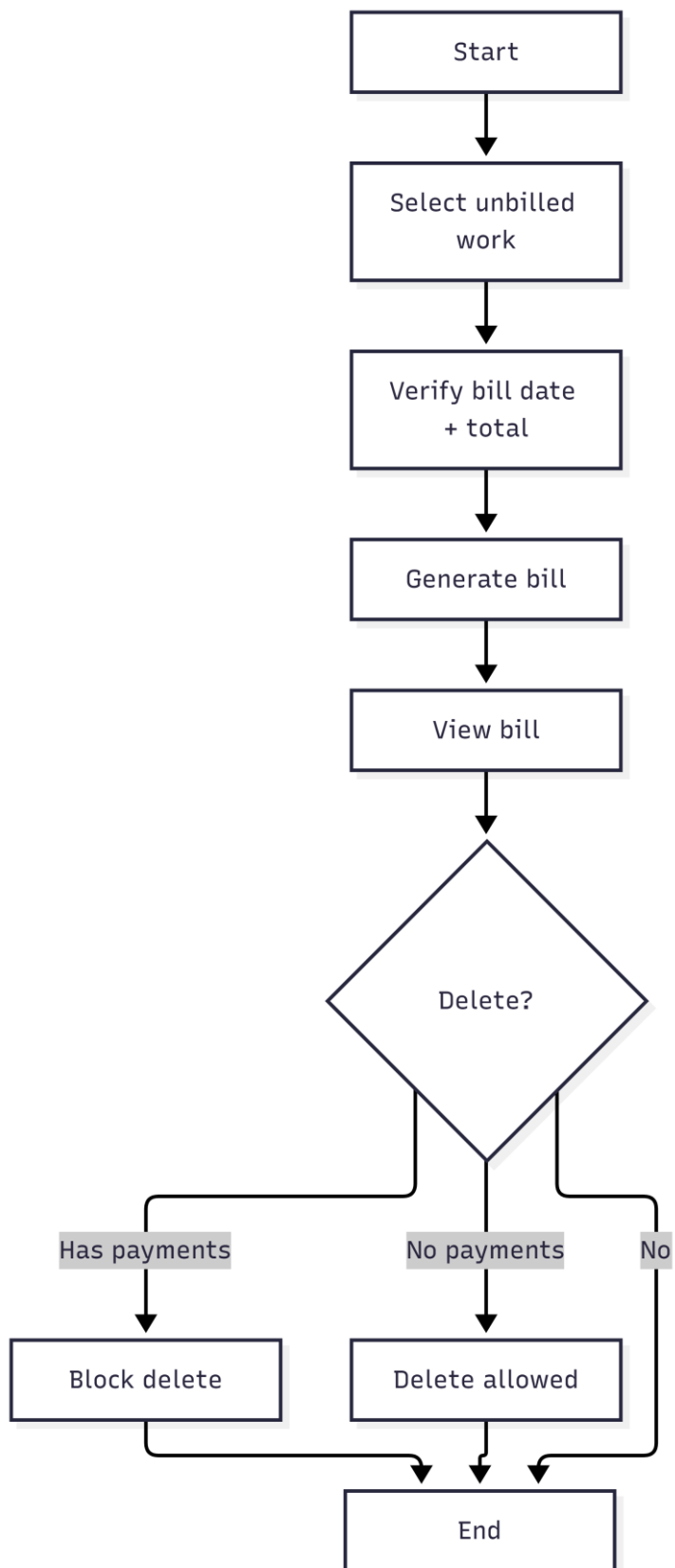
The Agricultural Work interface allows users to record different agricultural and irrigation activities with their related rates, area, duration, and amount.

4.2.6 Billing Management

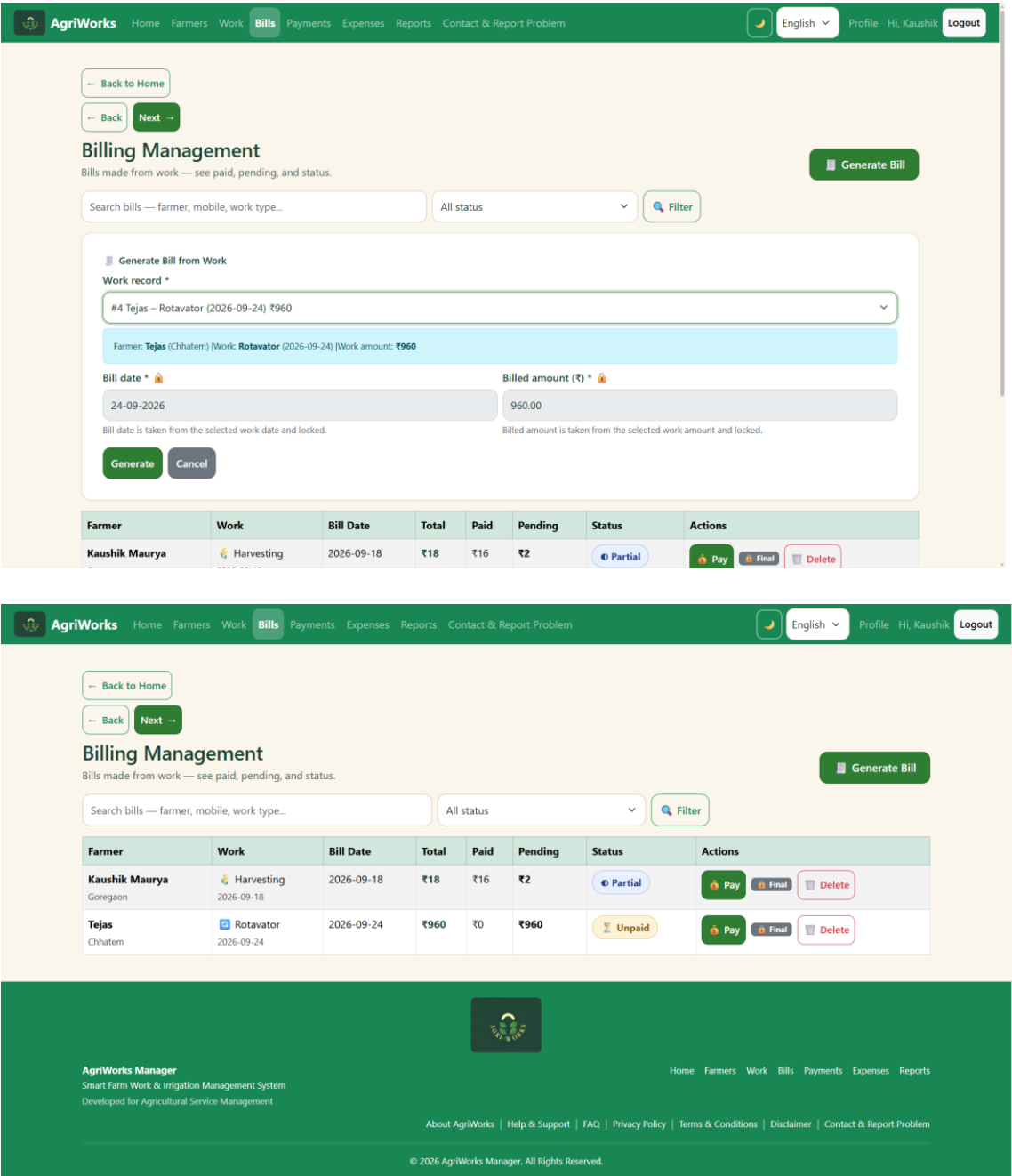
Algorithm:

1. Open Billing Management.
2. View available unbilled work.
3. Select a work record.
4. Verify the auto-filled bill date (locked).
5. Use the automatically calculated total amount.
6. Generate the bill.
7. View the generated bill.
8. Delete the bill if permitted.
9. End.

Flowchart:



Interface Diagram:



Validation:

A valid work record is required. Bill date cannot be in the future. Only one bill can be generated for a work record. Bill total is automatically taken from the work record and is locked. A bill with payments cannot be deleted.

Interface Description:

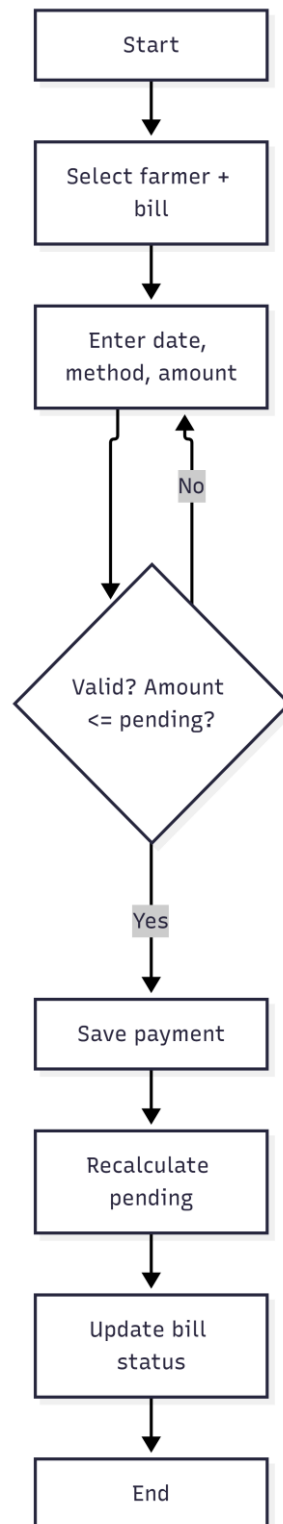
The Billing interface allows users to generate and manage bills for recorded agricultural work.

4.2.7 Payment Management

Algorithm:

1. Open Payment Management.
2. Search or select the farmer.
3. Select the related work and bill.
4. Enter payment date, method, and amount.
5. Validate the payment details.
6. Save the payment.
7. Recalculate the pending amount.
8. Update the bill status.
9. View or delete the payment if required.
10. End.

Flowchart:



Interface Diagram:

The interface is titled "Payment Management" and includes navigation links like "Back to Home", "Back", and "Next". It features a "Record Payment" button. The "Search Farmer" section has a search bar and a dropdown menu. The "Select Work / Bill" section shows a dropdown menu with details like "Rotavator | 24 Sept 2026 | 1.20 Acres | Total ₹960 | Pending ₹960". A summary table displays the bill status:

Total Bill	Paid	Remaining	Status
₹960	₹0	₹960	Unpaid

Below the summary, a message states "No payments recorded yet. Record the first payment for this bill." with a "Record Payment" button.

The bottom screenshot shows the interface after a payment of ₹150 has been recorded. The "Paid" amount is now ₹150, and the "Remaining" balance is ₹810. The status is updated to "Partial". A table lists the payment record:

Date	Method	Amount	Actions
2026-09-24	Cash	₹150	Locked Delete

The footer contains the "AgriWorks Manager" logo, a description of the system, and various links like "Home", "Farmers", "Work", "Bills", "Payments", "Expenses", and "Reports".

Validation:

Payment date cannot be in the future. Amount must be at least ₹1 and cannot exceed the pending amount. Paid bills cannot receive additional payments. Payment records cannot be edited.

Interface Description:

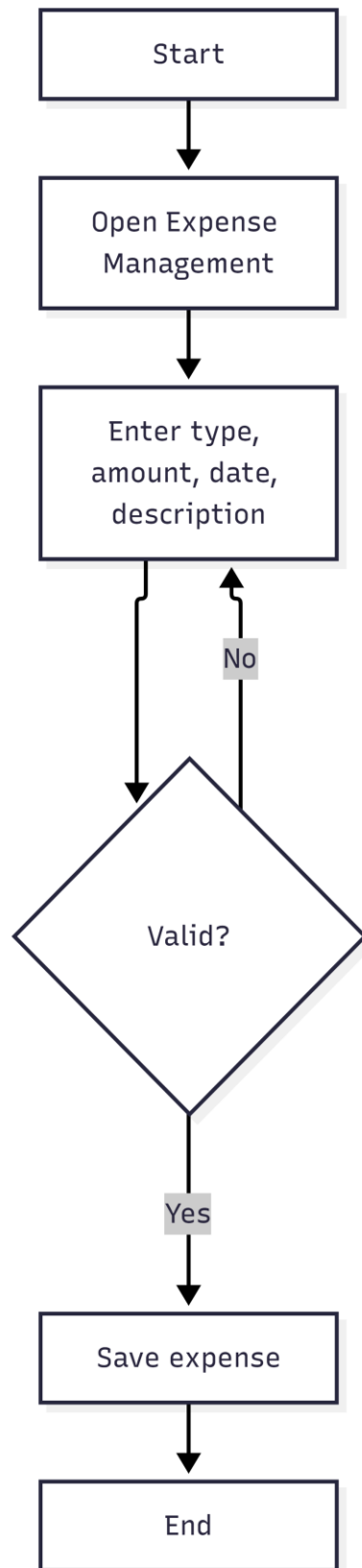
The Payment interface allows users to record payments against bills and automatically maintain their payment status.

4.2.8 Expense Management

Algorithm:

1. Open Expense Management.
2. View or search expense records.
3. Select the expense type.
4. Enter amount, date, and description.
5. Validate the entered information.
6. Save the expense.
7. View or delete the expense if required.
8. End.

Flowchart:



Interface Diagram:

The screenshot shows the 'Expense Management' section of the AgriWorks application. The top navigation bar is green with the AgriWorks logo and links to Home, Farmers, Work, Bills, Payments, Expenses (active), Reports, and Contact & Report Problem. On the right, there's a language dropdown set to English, a user profile for 'Hi, Kaushik', and a Logout button. The main content area has a light yellow background. It includes a 'Back to Home' button, 'Back' and 'Next' buttons, and the title 'Expense Management' with a subtitle 'Diesel, repairs, wages — write every kharcha here.' A green '+ Add Expense' button is in the top right. Below the title is a search bar 'Search expenses — type, note...', a dropdown menu set to 'All types', and a 'Filter' button. A status bar shows 'Total shown: ₹0 (0 records)'. The main area contains a message 'No expenses yet. Write your first kharcha above.' with a green '+ Add Expense' button. The footer is green, featuring the AgriWorks Manager logo, the text 'Smart Farm Work & Irrigation Management System' and 'Developed for Agricultural Service Management', and a secondary navigation bar with links to Home, Farmers, Work, Bills, Payments, Expenses, and Reports.

Validation:

Expense type and amount are required. Amount must be at least ₹1. Expense date cannot be in the future. Description is optional. Expense records cannot be edited.

Interface Description:

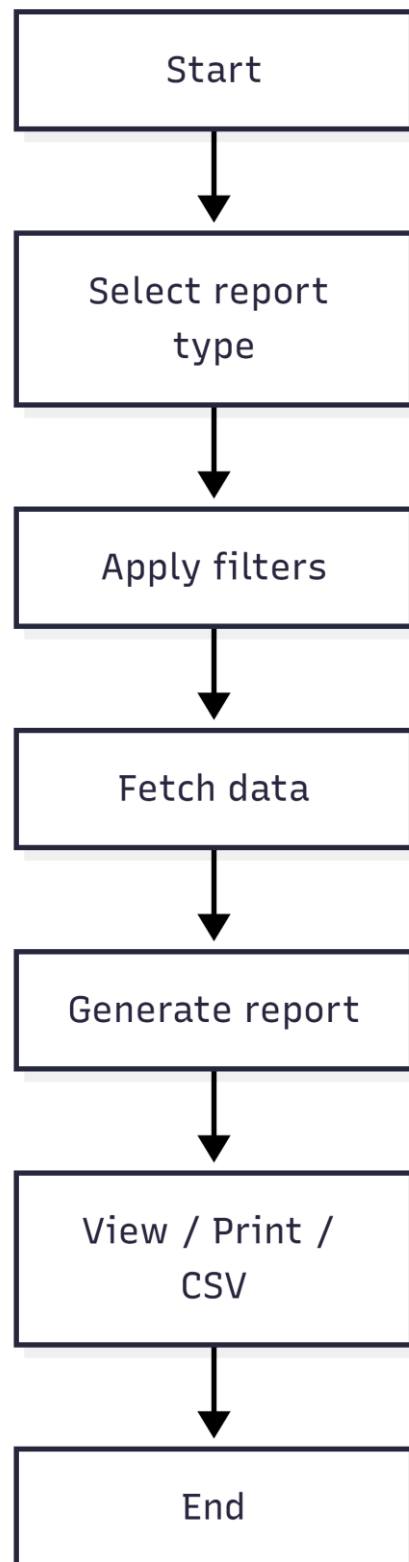
The Expense interface allows users to record and manage expenses such as diesel, maintenance, driver wages, and other expenses.

4.2.9 Reports

Algorithm:

1. Open the Reports section.
2. Select the required report type.
3. Apply the required filters.
4. Fetch the relevant user-specific data.
5. Generate the report.
6. Display the report summary and records.
7. Print or download the report if required.
8. End.

Flowchart:



Interface Diagram:

AgriWorks Home Farmers Work Bills Payments Expenses Reports Contact & Report Problem English Profile Hi, Kaushik Logout

Back to Home Back Next

Reports

Simple registers — pick a type, choose dates, print or save.

Agricultural Work Report Billing Report Payment Report Pending Payment Report Expense Report Business Performance Report

Search... All farmers From Date To Date dd-mm-yyyy dd-mm-yyyy Apply Print CSV

Pending Payment Report
Count: 2 | Total Pending: ₹842

ID	Farmer	Mobile	Village	Bill Date	Work Type	Total	Paid	Pending	Status
5	Tejas	6394636294	Chhatem	2026-09-24	Rotavator	₹960	₹120	₹840	Partial
4	Kaushik Maurya	7080395112	Goregaon	2026-09-18	Harvesting	₹18	₹16	₹2	Partial

Validation:

Only authenticated users can access reports. Applied filters must contain valid values and dates.

Interface Description:

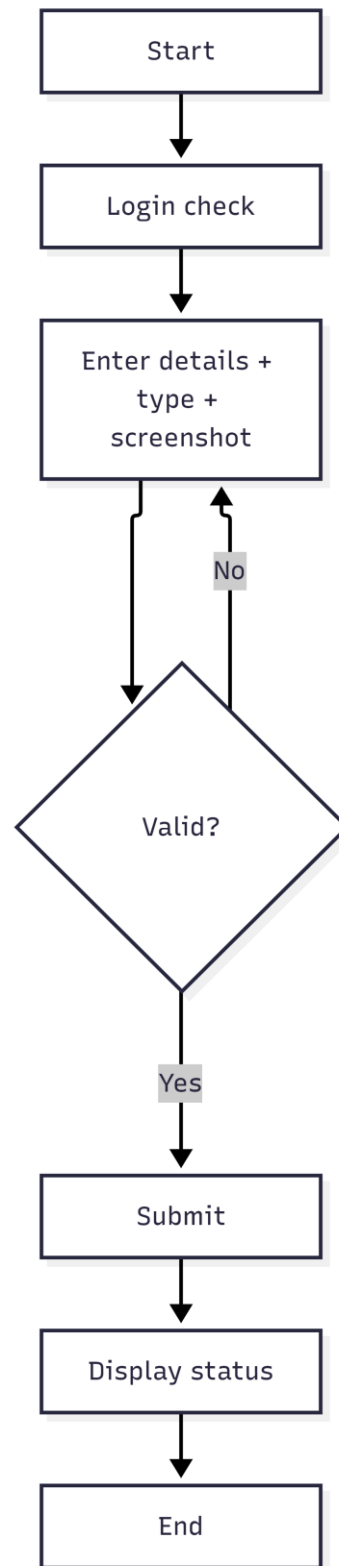
The Reports interface provides work, billing, payment, pending payment, expense, and performance reports with filtering, printing, and CSV download options.

4.2.10 Problem Report

Algorithm:

1. Open the Problem Report section.
2. Verify that the user is logged in.
3. Enter or review the problem details.
4. Select the problem type.
5. Add a screenshot (optional).
6. Validate the information.
7. Submit the report.
8. Display the submitted report and its status.
9. End.

Flowchart:



Interface Diagram:

The screenshot shows the 'Contact & Report Problem' page of the AgriWorks application. The top navigation bar is green and contains the AgriWorks logo, a menu with links to Home, Farmers, Work, Bills, Payments, Expenses, Reports, and 'Contact & Report Problem' (which is highlighted with a yellow box), a language dropdown set to English, and user profile information for 'Hi, Kaushik' with a Logout button. The main content area has a light beige background. It starts with a 'Contact Support' section containing a brief guide and links for 'Help & Support' and 'FAQ'. Below this is the 'Report a Problem' form. The form includes fields for 'Name *' (containing 'Kaushik') and 'Email *' (containing 'kaushikmaurya7318@gmail.com'). There is a 'Problem Type *' dropdown menu currently showing 'Select problem type'. A large text area for 'Description *' is provided. Below the description is a 'Screenshot (optional)' section with a 'Choose File' button and the text 'No file chosen'. A green 'Submit Problem' button is at the bottom of the form. At the very bottom of the page, there is a link for 'My Reports'.

Validation:

Login is required to submit a report. Name, email, problem type, and description are required. Email must be valid. Screenshot is optional and must satisfy the allowed file requirements.

Interface Description:

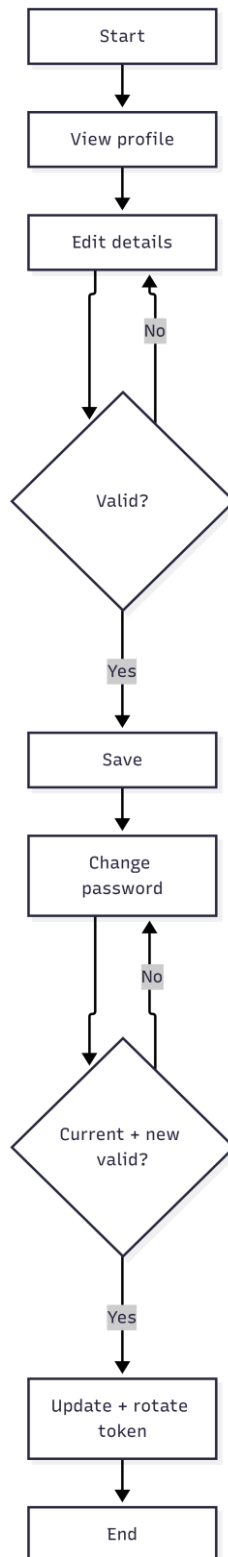
The Problem Report interface allows authenticated users to submit system problems and view the status of their submitted reports.

4.2.11 Profile and Change Password

Algorithm:

1. Open the Profile page.
2. View the user's profile information.
3. Edit the profile details if required.
4. Validate the entered information.
5. Save the profile changes.
6. Open the Change Password form.
7. Enter the current password, new password, and confirmation.
8. Validate the password details.
9. Update the password and rotate the authentication token.
10. End.

Flowchart:



Interface Diagram:

The screenshot displays the 'Profile' page of the AgriWorks application. The top navigation bar is green and contains the AgriWorks logo, a home icon, and links for Home, Farmers, Work, Bills, Payments, Expenses, Reports, and Contact & Report Problem. On the right side of the header, there is a language dropdown set to 'English', a 'Profile' button, a user greeting 'Hi, Kaushik', and a 'Logout' button. The main content area has a light beige background. At the top of this area is the title 'Profile' and a subtitle 'View and manage your account information.' Below this, there are two main sections. The first section, titled 'My Information', contains a table with the following details: Full Name (Kaushik), Last Name (Maurya), Company (R C ENTERPRISES), Email (kaushikmaurya7318@gmail.com), and Mobile (9555512182). An 'Edit Profile' button is located to the right of this table. The second section, titled 'Change Password', includes a 'Current Password *' field, a 'New Password *' field, and a 'Confirm New Password *' field. Below these fields is a note: 'Min 8 characters with uppercase, lowercase, number and special character.' and a 'Change Password' button. The bottom of the page features a green footer with a small circular logo.

Validation:

Full name, last name, and mobile number must be valid. Company is optional. Email is read-only. Password change requires the correct current password and a valid new password matching the confirmation.

Interface Description:

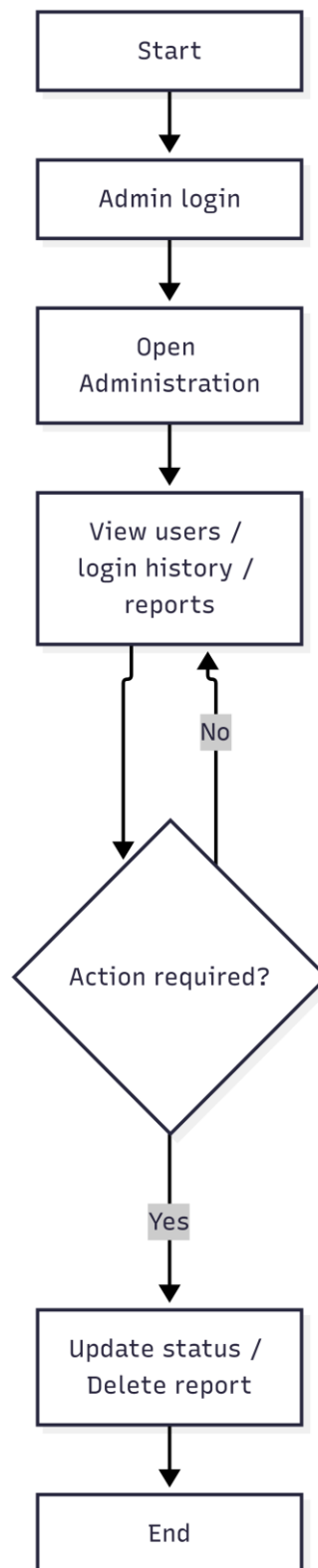
The Profile interface allows users to manage their personal information and change their account password securely.

4.2.12 Administration

Algorithm:

1. Administrator logs into the system.
2. Open the Administration section.
3. View the user registry and system overview.
4. View login history.
5. View and filter problem reports.
6. Select a problem report when action is required.
7. Update the problem report status or delete the report if authorized.
8. End.

Flowchart:



Interface Diagram

Admin Dashboard

Welcome,

Total Users: 7

Total Problem Reports: 2

Problem Reports

Pending: 1 | In Progress: 0 | Resolved: 1 | Total: 2

[View Problem Reports](#)

Recent Login Activity

User	Login Date	Login Time	Status
Kaushik	24 Sep 2026	09:42 PM	Successful
kaushikmaurya7318	24 Sep 2026	08:49 PM	Successful
kaushikmaurya7318	24 Sep 2026	01:44 AM	Successful
kaushikmaurya7318	24 Sep 2026	01:43 AM	Successful
kaushikmaurya7318	24 Sep 2026	01:34 AM	Successful

[View All Login History](#)

Recent Problem Reports

User	Problem Type	Date	Status	Action
kaushikmaurva8355	Login Problem	2026-09-20	Pending	View

User Overview

Search users...

Name	Username	Email	Mobile
ProdsmokeBFJGAH	prodsmoke159607	prodsmoke159607@example.com	900015
ProdsmokeBFJEHJ	prodsmoke159479	prodsmoke159479@example.com	900015
Swapnil	swapnilgawade1973	swapnilgawade1973@gmail.com	932669
bhagysfri	trivenimsuthar801	trivenimsuthar801@gmail.com	771084
Swastik	kaushikmaurya8355	kaushikmaurya8355@gmail.com	639463
—	Kaushik	kaushikmaurya96@gmail.com	—
Kaushik	kaushikmaurya7318	kaushikmaurya7318@gmail.com	955551

System Overview

API status: **ok** | Database status: **connected**

AgriWorks Manager
Smart Farm Work & Irrigation Management System
Developed for Agricultural Service Management

About AgriWorks | Help & Support | FAQ | Privacy Policy | Terms & Conditions | Disclaimer

© 2026 AgriWorks Manager. All Rights Reserved

Validation:

Only users with **is_superuser** permission can access Administration. Administrators cannot create users or modify/delete business records. Only problem report status update and problem report deletion are available as administrative actions.

Interface Description:

The Administration interface provides authorized administrators with system overview, user registry, login history, and problem report management functions.

CHAPTER 5: IMPLEMENTATION AND TESTING (OUTPUT SCREENS)

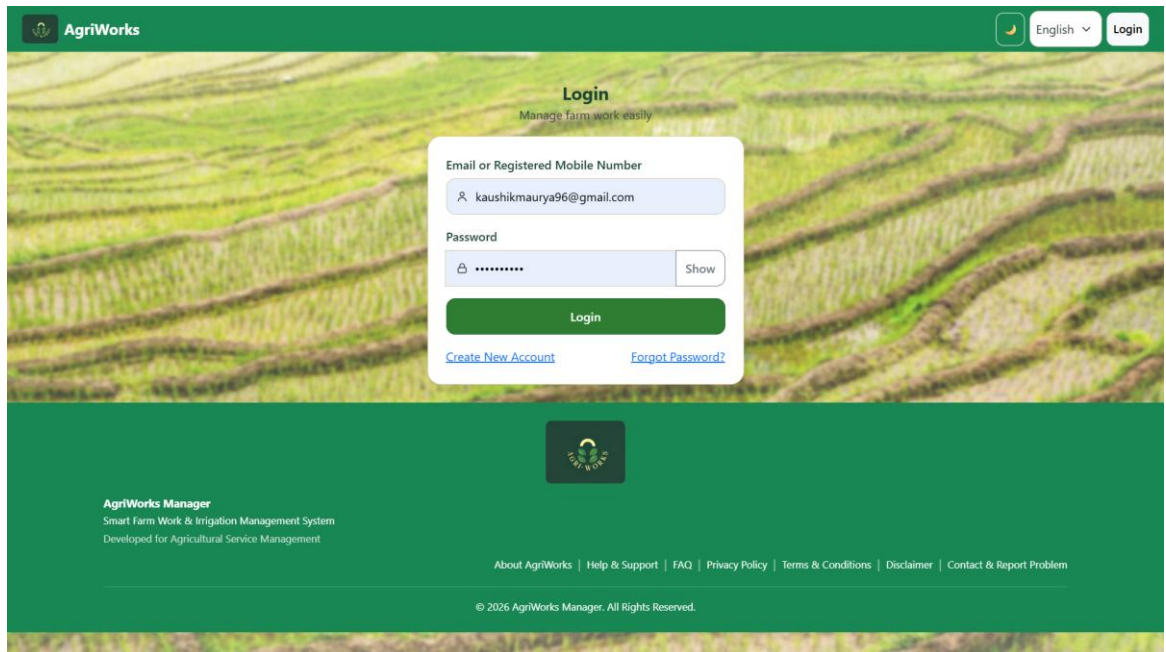
The application contains various pages to complete its functionalities such as user authentication, farmer management, agricultural work recording, billing, payment tracking, expense management, dashboard, reports, problem reporting, and administration.

Following are the code and screenshots for the various pages. Each section shows the output screen, its explanation, and the key source code with colour formatting. Business logic such as automatic amount calculation, duplicate guards, locked bill totals, and payment status updates is enforced on the server side. The frontend is developed in React.js, backend APIs in Django REST Framework, and data is stored in MySQL.

5.1 Implementation Approaches

The system is implemented as a React.js frontend communicating with a Django REST Framework backend over token-authenticated JSON APIs, with MySQL as the database. Development was done module-wise in the order Authentication, Farmer, Agricultural Work, Billing, Payment, Expense, Dashboard and Reports, Problem Reporting, and Administration. Business logic such as amount calculation, duplicate guards, locked bill totals, and payment status recalculation is enforced server-side. The frontend is deployed on Vercel, backend on Render, and database on Aiven.

1)Auth - accounts/views.py + Login.jsx / Signup.jsx / ForgotPassword.jsx



The login page features a green header with the AgriWorks logo, a language selector set to English, and a Login button. The main content area has a background image of a terraced field. A central white login form contains fields for 'Email or Registered Mobile Number' (with the value 'kaushikmaurya96@gmail.com') and 'Password' (masked with dots). A 'Show' button is next to the password field. Below the fields is a green 'Login' button. Links for 'Create New Account' and 'Forgot Password?' are at the bottom of the form. The footer includes the AgriWorks Manager logo, a description of the system, and a list of links: About AgriWorks, Help & Support, FAQ, Privacy Policy, Terms & Conditions, Disclaimer, and Contact & Report Problem. A copyright notice for 2026 is also present.

AgriWorks English Login

Login
Manage farm work easily

Email or Registered Mobile Number
kaushikmaurya96@gmail.com

Password
•••••••• Show

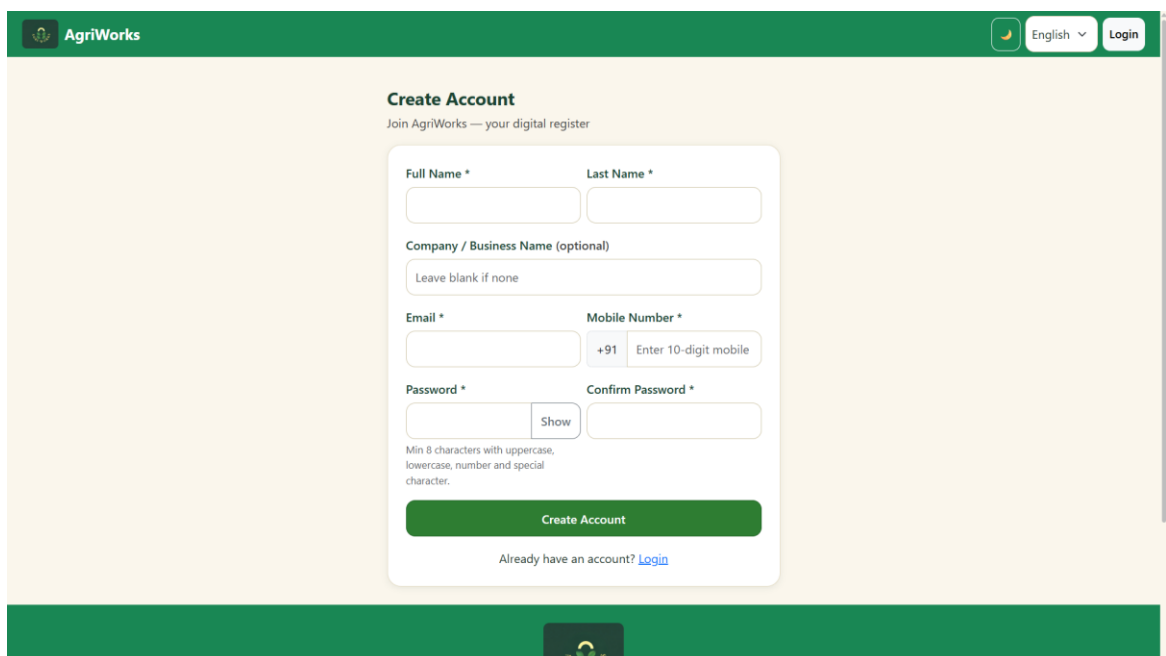
Login

[Create New Account](#) [Forgot Password?](#)

AgriWorks Manager
Smart Farm Work & Irrigation Management System
Developed for Agricultural Service Management

[About AgriWorks](#) | [Help & Support](#) | [FAQ](#) | [Privacy Policy](#) | [Terms & Conditions](#) | [Disclaimer](#) | [Contact & Report Problem](#)

© 2026 AgriWorks Manager. All Rights Reserved.



The create account page has a green header with the AgriWorks logo, a language selector set to English, and a Login button. The main content area has a light beige background. A central white form titled 'Create Account' with the subtitle 'Join AgriWorks — your digital register' contains several fields: 'Full Name *' and 'Last Name *' (both empty), 'Company / Business Name (optional)' (with a hint 'Leave blank if none'), 'Email *' (empty), 'Mobile Number *' (with a '+91' prefix and a hint 'Enter 10-digit mobile'), 'Password *' (empty, with a 'Show' button), and 'Confirm Password *' (empty). A green 'Create Account' button is at the bottom of the form. Below the button is a link for 'Already have an account? Login'. The footer includes the AgriWorks Manager logo.

AgriWorks English Login

Create Account
Join AgriWorks — your digital register

Full Name * Last Name *
Company / Business Name (optional)
Leave blank if none

Email * Mobile Number *
+91 Enter 10-digit mobile

Password * Confirm Password *
Show

Min 8 characters with uppercase, lowercase, number and special character.

Create Account

Already have an account? [Login](#)

- Registration with full name, last name, optional company, email, 10-digit mobile, strong password with Show toggle.
- Login with email/username/registered mobile + password with 24-hour token expiry and single active session.
- Password reset via single-use 24-hour link; unknown emails get same generic message.
- Validations: email format, mobile 10-digit 6-9 start, password 8 chars upper/lower/digit/special, confirm match.
- Security: rate-limited requests, old tokens invalidated on reset, history on success.
- User-friendly: inline errors, Hindi/Marathi, Light/Dark, auto-login after registration.

//accounts/views.py

```
# accounts/views.py - Step 1: email a single-use reset link
@api_view(['POST'])
@permission_classes([AllowAny])
@throttle_classes([PasswordResetRateThrottle])
def password_reset_request_view(request):
    email = (request.data.get('email') or '').strip().lower()
    if not email:
        return Response({'error': 'Email is required.'},
                        status=status.HTTP_400_BAD_REQUEST)
    generic_response = {'message': 'If an account exists with this email '
                                'address, a password reset link has '
                                'been sent.'}
```

```

try:
    user = User.objects.get(email__iexact=email)
except (User.DoesNotExist, User.MultipleObjectsReturned):
    return Response(generic_response)
uid = urlsafe_base64_encode(force_bytes(user.pk))
token = token_generator.make_token(user)
try:
    send_mail(subject, body, settings.DEFAULT_FROM_EMAIL, [user.email],
              fail_silently=False)
except Exception:
    logger.warning('Password reset email could not be sent.')
    return Response(
        {'error': 'Could not send the password reset email.'},
        status=status.HTTP_500_INTERNAL_SERVER_ERROR)
return Response(generic_response)

# Step 2: set a new password using the uid/token from the link
def password_reset_confirm_view(request):
    uid = request.data.get('uid') or ''
    token = request.data.get('token') or ''
    new_password = request.data.get('new_password') or ''
    confirm_password = request.data.get('confirm_password') or ''
    if not uid or not token:
        return Response({'error': 'Reset link is invalid or expired.'},
                        status=status.HTTP_400_BAD_REQUEST)
    if new_password != confirm_password:
        return Response({'error': 'Passwords do not match.'},
                        status=status.HTTP_400_BAD_REQUEST)
    user = User.objects.get(pk=force_str(urlsafe_base64_decode(uid)))
    if not token_generator.check_token(user, token):
        return Response({'error': 'Reset link is invalid or expired.'},
                        status=status.HTTP_400_BAD_REQUEST)
    user.set_password(new_password)
    user.save()
    Token.objects.filter(user=user).delete() # single-use: old sessions end
    return Response({'message': 'Password has been reset. Please log in.'})

```

2) Farmers.jsx / farmers/views.py

The top screenshot displays the 'Farmer Management' page. It includes a navigation bar with links like Home, Farmers, Work, Bills, Payments, Expenses, Reports, and Contact & Report Problem. A search bar is present with the text 'Search Farmers — name, mobile, village...'. Below the search bar is a table with columns: Name, Mobile, Village, Address, and Actions. The table contains two rows: Tejas (Mobile: 6394636294, Village: Chhatem, Address: Barahani Block of Chandauli District, Uttar Pradesh (Pin Code: 232110)) and Kaushik Maurya (Mobile: 7080395112, Village: Goregaon, Address: Film City). Each row has 'Edit' and 'Delete' buttons. A '+ Add Farmer' button is in the top right.

The bottom screenshot shows the 'Add Farmer' form. It has fields for 'Farmer name *', 'Mobile (10 digits) *', 'Village *', and 'Address'. The 'Mobile' field has a '+91' prefix and a placeholder 'Enter 10-digit mobile number'. The 'Address' field has a placeholder 'House / street (optional)'. There are 'Save' and 'Cancel' buttons. Below the form is a table with the same data as the top screenshot. A '+ Add Farmer' button is also present in the top right.

- Manage farmer records with name, 10-digit mobile, village, optional address, search by name/mobile/village.
- Data restricted to logged-in user; deletion of farmers having works blocked to protect bills and payments.
- Inline validation errors with instant search filters.
- Security: IsAuthenticated APIs, ownership enforced in queryset.
- Delete guard keeps farmer and all linked records untouched on block.
- User-friendly single-page register with Add/Edit inline.

//farmers/views.py

```
# farmers/views.py - Farmer CRUD API (requires login)
class FarmerViewSet(viewsets.ModelViewSet):
    serializer_class = FarmerSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        # Each user sees only their own farmers (User -> Farmer)
        qs = Farmer.objects.filter(user=self.request.user)
        search = (self.request.query_params.get('search') or '').strip()
        if search:
            qs = qs.filter(
                Q(name__icontains=search)
                | Q(mobile__icontains=search)
                | Q(village__icontains=search)
            )
        return qs

    def perform_create(self, serializer):
        serializer.save(user=self.request.user)

    def perform_destroy(self, instance):
        # Delete guard: a farmer with work records keeps everything
        if instance.works.exists():
            raise serializers.ValidationError(
                'Cannot delete this farmer because work records exist.')
        instance.delete()
```

3) Works.jsx / works/serializers.py

The screenshot shows the 'Work' form in the AgriWorks application. The form is structured as follows:

- 1. Farmer:** A dropdown menu labeled 'Farmer *' with the placeholder 'Select farmer'.
- 2. Work Type:** A row of radio buttons for 'Ploughing', 'Rotavator', 'Cultivation', 'Harvesting', 'Irrigation', 'Other', and 'Land Leveling'. Below them is a 'Work type *' dropdown with the placeholder 'Select type'.
- 3. Date & Place:** A 'Work date *' field with the value '27-09-2026' and a 'Field / Location *' field with the value 'e.g. North field, Gat No. 12'.
- 4. Work Details:** An 'Area (acres) *' field with the value '0.0'.
- 5. Amount:** An 'Amount (₹) *' field with a currency symbol '₹'.

At the bottom of the form are 'Save' and 'Cancel' buttons.

- Records Ploughing, Rotavator, Cultivation, Harvesting, Irrigation, Land Leveling and Other with date, location, remark and work description.
- Auto-calculates area charges (area x rate) and irrigation charges ((hours + minutes/60) x rate); rejects future dates and duplicates.
- Saved records locked (no PUT/PATCH); delete blocked if bill generated.
- Irrigation needs hours/minutes/rate; Other needs description + manual amount.
- Concurrent duplicate race converted to friendly 400 via IntegrityError catch.
- User-friendly type-wise dynamic fields with inline validation.

//works/serializers.py

```
# works/serializers.py - irrigation: (hours + minutes/60) x rate
total = (Decimal(hours) + Decimal(minutes) / Decimal(60)) * Decimal(rate)
attrs['work_description'] = None

# area-based work: area x rate per acre
attrs['amount'] = (Decimal(area) * Decimal(rate)).quantize(
    Decimal('0.01'))

def validate_work_date(self, value):
    if value > date.today():
        raise serializers.ValidationError(
            'Work date cannot be in the future.')
    return value

# works/models.py - duplicate guard: same user + farmer + type + date
constraints = [
    models.UniqueConstraint(
        fields=['user', 'farmer', 'work_type', 'work_date'],
        name='unique_work_per_farmer_type_date',
```

```

    ),
]

# works/views.py - saved records are locked (no PUT/PATCH)
http_method_names = ['get', 'post', 'delete', 'head', 'options']

def perform_create(self, serializer):
    # concurrent duplicate race becomes the same friendly 400
    try:
        with transaction.atomic():
            serializer.save(user=self.request.user)
    except IntegrityError:
        raise serializers.ValidationError({
            'work_date': 'A work entry for this work type already exists on
this date.'})

def perform_destroy(self, instance):
    # billed work is the source of truth - keep work, bill, payments
    if hasattr(instance, 'bill'):
        raise serializers.ValidationError(
            'Cannot delete this work because a bill has already '
            'been generated for it.')
    instance.delete()

```

4) Bills.jsx / billing/views.py

Billing Management
Bills made from work — see paid, pending, and status.

Search bills — farmer, mobile, work type... All status Filter

Generate Bill from Work

Work record *

Select unbilled work

No unbilled works. Add work first in Work page.

Bill date * dd-mm-yyyy

Billed amount (₹) *

Bill date is taken from the selected work date and locked.

Billed amount is taken from the selected work amount and locked.

Generate Cancel

Farmer	Work	Bill Date	Total	Paid	Pending	Status	Actions
Kaushik Maurya Goregaon	Harvesting 2026-09-18	2026-09-18	₹18	₹16	₹2	Partial	Pay Final Delete
Tejas Chhatem	Rotavator 2026-09-24	2026-09-24	₹960	₹120	₹840	Partial	Pay Final Delete

- Generates bills only from unbilled work; date and total auto-filled from work and locked.
- Only one bill per work (OneToOne + row lock); deletion blocked if payments exist.
- Status Unpaid/Partial/Paid auto with payments; filters by farmer and status.
- paid_sum annotated once per page (no N+1 queries).
- Malformed farmer filter fails loudly with 400.
- User-friendly one-click generate with lock badges.

//billing/views.py

```
# billing/views.py — generated bills are finalized (no PUT/PATCH)
class BillViewSet(viewsets.ModelViewSet):
    serializer_class = BillSerializer
    permission_classes = [IsAuthenticated]
    http_method_names = ['get', 'post', 'delete', 'head', 'options']

    def perform_create(self, serializer):
        # lock the work row so concurrent creations serialize
        with transaction.atomic():
            locked_work = Work.objects.select_for_update().get(
                id=work.id, user=self.request.user)
            if hasattr(locked_work, 'bill'):
                raise serializers.ValidationError({
                    'work': 'Bill already exists for this work record.'})
            serializer.save(user=self.request.user)

    def perform_destroy(self, instance):
        # a bill with payments is the audit trail — keep it
        if instance.payments.exists():
```

```

        raise serializers.ValidationError(
            'Cannot delete this bill because payment records exist.')
    instance.delete()

    def get_queryset(self):
        # paid sums annotated once per page (no N+1 per bill)
        qs = Bill.objects.select_related('farmer', 'work').filter(
            user=self.request.user).annotate(paid_sum=Sum('payments__amount'))
        status = (self.request.query_params.get('status') or '').strip()
        if status in ('Unpaid', 'Partial', 'Paid'):
            qs = qs.filter(status=status)
        return qs

```

5) Payments.jsx / payments/views.py

- Records Cash/UPI/Bank Transfer/Cheque/Other with Total/Paid/Remaining/Status cards always visible.
- Blocks amounts exceeding pending; Paid bills disabled for new payments.
- Recalculates Unpaid/Partial/Paid on every save and delete under bill row-lock.
- Saved payments locked (no PUT/PATCH); corrections via delete with auto refresh.
- Whole rupees Rs 1+, future dates rejected, overpayment re-checked atomically.
- User-friendly farmer→bill chain with remaining-pay hints

//payments/views.py

```
# payments/views.py - Paid/Partial/Unpaid from total vs sum(payments)
def update_bill_status(bill):
    paid = bill.payments.aggregate(total=Sum('amount'))['total'] or
    Decimal('0')
    if paid <= 0:
        bill.status = 'Unpaid'
    elif paid >= bill.total_amount:
        bill.status = 'Paid'
    else:
        bill.status = 'Partial'
    bill.save(update_fields=['status', 'updated_at'])

class PaymentViewSet(viewsets.ModelViewSet):
    serializer_class = PaymentSerializer
    permission_classes = [IsAuthenticated]
    # no PUT/PATCH: saved payments are locked, delete refreshes status
    http_method_names = ['get', 'post', 'delete', 'head', 'options']
```

```

def perform_destroy(self, instance):
    with transaction.atomic():
        locked_bill = Bill.objects.select_for_update().get(
            id=instance.bill_id, user=self.request.user)
        instance.delete()
        update_bill_status(locked_bill)

def perform_create(self, serializer):
    # re-check under row lock: reject overpayment with the same 400
    with transaction.atomic():
        locked_bill = Bill.objects.select_for_update().get(
            id=instance.bill_id, user=self.request.user)
        pending = bill_pending(locked_bill, exclude_payment_id=None)
        amount = serializer.validated_data.get('amount')
        if amount is not None and Decimal(amount) > pending:
            raise serializers.ValidationError({
                'amount': f'Payment Rs {amount} exceeds remaining Rs
{pending}.'})
        payment = serializer.save(user=self.request.user)
        update_bill_status(locked_bill)

```

6) Expenses.jsx / expenses/serializers.py

- Records Diesel/Maintenance/Driver Wages/Other with amount, date, optional note + search and type filter.
- Only whole rupees Rs 1+; future dates rejected; records locked (no PUT/PATCH).
- Types match real farm costs; messages guide entry.
- Server validates even direct API calls; Asia/Kolkata local date used.
- Whole values like 2000.00 pass as whole; decimals rejected.
- User-friendly one-form add with running total bar.

//expenses/serializers.py

```
# expenses/serializers.py — only positive whole rupees from Rs 1
def validate_amount(self, value):
    try:
        dec = Decimal(value)
    except Exception:
        raise serializers.ValidationError(
            'Amount must be a whole number of at least Rs 1.')
    if not dec.is_finite() or dec < 1 or dec != dec.to_integral_value():
        raise serializers.ValidationError(
            'Amount must be a whole number of at least Rs 1.')
    return value

def validate_date(self, value):
    if value > timezone.localdate():
        raise serializers.ValidationError(
            'Date cannot be in the future.')
    return value
```

7) Dashboard.jsx + Reports.jsx / api/views.py

The image shows two screenshots of the AgriWorks application. The top screenshot is the dashboard for user Kaushik, displaying various metrics and recent activities. The bottom screenshot is the 'Reports' page, showing filters and a table of agricultural work records.

Dashboard Metrics:

- Work Records: 2 (Total jobs done)
- Total Income (billed): ₹978 (Billed so far)
- Payments Received: ₹136 (Money collected)
- Pending Payments: ₹842 (Still to collect)
- Total Expenses: ₹2000 (Money spent)
- Farmers: 2 (Farmers in register)

Recent Work:

- Tejas - Rotavator (2026-09-24) ₹960
- Kaushik Maurya - Harvesting (2026-09-18) ₹18

Recent Payments:

- Tejas - Bill #5 (2026-09-24) Cash ₹120
- Kaushik Maurya - Bill #4 (2026-09-18) Cash ₹16

Recent Expenses:

- Diesel (2026-09-27) ₹2000

Reports Page:

Simple registers — pick a type, choose dates, print or save.

Filters: Search..., All farmers, From Date (dd-mm-yyyy), To Date (dd-mm-yyyy), All work types.

Agricultural Work Report
Count: 2 | Total Area: 2.70 | Total Amount: ₹978

ID	Farmer	Village	Work Type	Date	Field / Location	Remark	Work Description	Area	Amount	Hours	Minutes	Rate per Hour	Rate per Acre
4	Tejas	Chhatem	Rotavator	2026-09-24	North Field	NA		1.20	₹960				₹800
3	Kaushik Maurya	Goregaon	Harvesting	2026-09-18	Infinity Mall			1.50	₹18				₹12

- Shows work/income/payments/pending/expenses/farmers totals with recent activities; six reports with filters, print, CSV.
- Only own data; Paid/Partial/Unpaid counts computed server-side.
- Cards + Hindi/Marathi + Light/Dark one-click review.
- Reports: work/billing/payment/pending/expense/performance with date/farmer/status filters.
- Recent top-5 reports feed the dashboard; paginated login history for admin.

- User-friendly summary-first layout with CSV for accountants.

//api/views.py

```
# api/views.py - system overview for the Admin Dashboard
total_users = User.objects.count()
reports = ProblemReport.objects.all()
return Response({
    'totals': {
        'users': total_users,
        'problem_reports': reports.count(),
    },
    'problem_reports': {
        'pending': reports.filter(status='Pending').count(),
        'in_progress': reports.filter(status='In Progress').count(),
        'resolved': reports.filter(status='Resolved').count(),
    },
    'system': {'api': 'ok', 'database': db_status},
})

# recent reports for the dashboard (newest first, top 5)
recent_reports = ProblemReport.objects.select_related('user').order_by(
    '-created_at', '-id')[:5]
'recent_reports': [
    {
        'id': r.id,
        'username': r.user.username if r.user_id else '',
        'problem_type': r.problem_type,
        'status': r.status,
    }
    for r in recent_reports
]
```

8) ContactSupport.jsx / support/views.py

Date	Problem Type	Description	Status
2026-09-27	Dashboard Problem	NA	Pending

- Submits 8 problem types with screenshot up to 2MB; tracks Pending/In Progress/Resolved in My Reports.
- New reports forced Pending under own account; users see only theirs.
- Only admins update/delete; screenshots owner-or-admin only with same-404 (no existence leak).
- Uploads magic-byte gated, nosniff inline serve, extension allowlist.
- Status tracking removes follow-up calls.
- User-friendly one-page submit + track.

//support/views.py

```
# support/views.py - AgriWorks admin = is_superuser (not is_staff)
class IsSuperUser(BasePermission):
    def has_permission(self, request, view):
        return bool(request.user and request.user.is_authenticated
                    and request.user.is_superuser)

class ProblemReportViewSet(viewsets.ModelViewSet):
    serializer_class = ProblemReportSerializer

    def get_queryset(self):
        qs = ProblemReport.objects.select_related('user').all()
        if self.request.user.is_superuser:
            return qs
        return qs.filter(user=self.request.user)

    def get_permissions(self):
        if self.action in ('update', 'partial_update', 'destroy'):
            return [IsSuperUser()]
        return [IsAuthenticated()]
```

```

def perform_create(self, serializer):
    serializer.save(user=self.request.user, status='Pending')

# screenshots: owner or superuser only, same 404 otherwise (no leak)
report = ProblemReport.objects.select_related('user').get(
    screenshot='problem_screenshots/' + name)
user = request.user
if not (user.is_superuser or report.user_id == user.id):
    return _screenshot_missing()
response = FileResponse(open(field_file.path, 'rb'),
    content_type=content_type)
response['X-Content-Type-Options'] = 'nosniff'
response['Content-Disposition'] = 'inline'
return response

```

9) AdminDashboard.jsx / api/views.py + support/views.py

AgriWorks Admin Dashboard

English | Profile | Hi | Logout

Total Users: 7 | Total Problem Reports: 3

Problem Reports: Pending: 2 | In Progress: 0 | Resolved: 1 | Total: 3

[View Problem Reports](#)

Recent Login Activity

User	Login Date	Login Time	Status
Kaushik	27 Sep 2026	02:57 PM	Successful
kaushikmaurya7318	27 Sep 2026	02:02 PM	Successful
kaushikmaurya7318	27 Sep 2026	01:26 PM	Successful
Kaushik	26 Sep 2026	10:55 PM	Successful
kaushikmaurya7318	26 Sep 2026	08:03 PM	Successful

[View All Login History](#)

Recent Problem Reports

User	Problem Type	Date	Status	Action
kaushikmaurya7318	Dashboard Problem	2026-09-27	Pending	View
kaushikmaurya8355	Login Problem	2026-09-20	Pending	View
kaushikmaurya8355	Report Problem	2026-09-20	Resolved	View

User Overview

Search users...

Name	Username	Email	Mobile	Role
—	prodsmoke159607	prodsmoke159607@example.com	—	User
—	prodsmoke159479	prodsmoke159479@example.com	—	User
Swapnil	swapnilgawade1973	swapnilgawade1973@gmail.com	9326699619	User
bhagysri	trivenimsuthar801	trivenimsuthar801@gmail.com	7710844801	User
Swastik	kaushikmaurya8355	kaushikmaurya8355@gmail.com	6394636294	User
—	Kaushik	kaushikmaurya96@gmail.com	—	Admin
Kaushik	kaushikmaurya7318	kaushikmaurya7318@gmail.com	9555512182	User

System Overview

API status: **ok** | Database status: **connected**

AgriWorks Manager
Smart Farm Work & Irrigation Management System
Developed for Agricultural Service Management

About AgriWorks | Help & Support | FAQ | Privacy Policy | Terms & Conditions | Disclaimer

© 2026 AgriWorks Manager. All Rights Reserved

- Superuser overview, searchable registry, login history with date/time/IP/browser, all reports with update/delete.
- is_superuser only (is_staff alone = normal user); no user creation or business-record edits.
- Safe fields only — never passwords/hashes/tokens.
- History newest-first with pagination; deleted users retained as audit rows.
- Token-authenticated APIs behind cards and tables.

➤ User-friendly single-dashboard monitoring.

//api/views.py + support/views.py

```
# support/views.py - AgriWorks admin = is_superuser (not is_staff)
class IsSuperUser(BasePermission):
    def has_permission(self, request, view):
        return bool(request.user and request.user.is_authenticated
                    and request.user.is_superuser)

# api/views.py - registered-users registry (superuser only)
# same safe fields only - never passwords, hashes, or tokens
all_users = User.objects.order_by('-date_joined', '-id')
'users': [
    {
        'username': u.username,
        'full_name': getattr(getattr(u, 'profile', None), 'full_name', '') or
'',
        'email': u.email,
        'mobile': getattr(getattr(u, 'profile', None), 'mobile', '') or '',
        'is_active': u.is_active,
        'is_superuser': u.is_superuser,
    }
    for u in all_users
]

# admin login history (newest first, login date + time formatted)
records = LoginHistory.objects.select_related('user').order_by(
    '-created_at', '-id')
items = []
for h in records:
    logged_at = timezone.localtime(h.created_at) if h.created_at else None
    items.append({
        'username': h.user.username if h.user_id else '',
        'login_date': logged_at.strftime('%d %b %Y') if logged_at else '',
        'login_time': logged_at.strftime('%I:%M %p') if logged_at else '',
        'status': h.status,
        'ip_address': h.ip_address or '',
    })
```

Deployment: frontend on Vercel, backend on Render with EMAIL_TIMEOUT=8 for safe SMTP, database on Aiven MySQL. Responsive on computer, tablet and mobile with English/Hindi/Marathi and Light/Dark mode. Testing and test reports are covered in Chapter 6.

CHAPTER 6: TEST CASES AND TEST REPORTS

Testing was carried out module-wise to verify that each functionality works as per requirements and that invalid inputs are safely rejected. Functional testing covers authentication, farmer, work, billing, payment, expense, dashboard, reports, problem reporting, and administration modules. Security testing covers token authentication, ownership checks, and server-side guards. Backend automated tests and manual UI testing on Chrome/Edge were both performed. The following tables list the test cases with steps, data, expected results, and status.

6.1 Module Testing

>>Login module

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
TC_1	Login Page	Verify valid login	1)Enter email 2)Enter password 3)Click Login	Valid email + valid password	Dashboar displays	Home page shown	Pass
TC_2	Login Page	Verify invalid email login	1)Enter invalid email 2)Enter password	Invalid email + valid password	Invalid credentials message	Login stays, no history row	Pass
TC_3	Login Page	Verify invalid password login	1)Enter email 2)Enter wrong password	Valid email + invalid password	Invalid credentials message	Login stays, no history row	Pass
TC_4	Login Page	Verify blank submit	1)Click Login without data	Blank fields	Enter all details message	Validation shown, stays on page	Pass

TC_5	Forgot link	Verify Forgot link navigation	1)Click Forgot Password link	—	Forgot Password page opens	Navigation done	Pass
TC_6	Show Password	Verify show password toggle	1)Click Show	Show toggle	Password visible	Toggle works	Pass

>>Forgot password

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
TC_1	Password reset request	Verify registered email request	1)Enter email 2)Send	Registered email	Generic success message	Reset mail sent	Pass
TC_2	Unknown email request	Verify unknown email handling	1)Enter unknown 2)Send	Unknown email	SAME generic message	No mail sent, no leak	Pass
TC_3	Invalid email format	Verify bad format rejection	1)Enter bad email 2)Send	Bad email format	Valid email error	Stays, validation shown	Pass
TC_4	Password reset confirm	Verify matching passwords reset	1)Open link 2)Set same passwords	Matching strong passwords	Password reset done	Logged out, login asked	Pass
TC_5	Confirmation mismatch	Verify mismatched passwords	1)Enter different passwords	Mismatched passwords	Passwords do not match	Reset blocked, stays	Pass
TC_6	Expired link handling	Verify used link rejection	1)Open used link	Used token	Invalid or expired error	Reset blocked, stays	Pass

>>Farmer module

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
TC_1	Farmer list	Verify farmer list view	1)Login 2)Open Farmers	—	Own farmers listed	List displayed	Pass
TC_2	Add farmer	Verify valid farmer add	1)Add details 2)Save	Valid name + mobile + village	Farmer saved	New row in list	Pass
TC_3	Invalid mobile validation	Verify short mobile rejection	1)Enter short mobile 2)Save	9-digit mobile	Mobile error	Stays, save blocked	Pass
TC_4	Farmer search	Verify keyword search	1)Type keyword 2)Search	Search keyword	Filtered records	Correct rows shown	Pass
TC_5	Farmer delete guard	Verify delete blocked with works	1)Delete farmer having works	Farmer having works	Cannot delete message	Farmer + records kept	Pass
TC_6	Farmer delete	Verify free farmer delete	1)Delete free farmer	Farmer without works	Farmer deleted	Row removed	Pass

>>Agricultural Work module

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
--------------	----------	-----------	------------	-----------	-----------------	----------------	--------------------

TC _1	Area-based billing	Verify area charge calc	1)Select farmer+type 2)Enter area/rate 3)Save	Area 1.5 + rate 1000	Amount 1500 auto	Work saved in list	Pass
TC _2	Irrigation billing	Verify duration charge calc	1)Select Irrigation 2)Enter duration +rate 3)Save	2h30m + hourly rate	2.5 x rate auto	Work saved in list	Pass
TC _3	Invalid area validation	Verify zero/negative rejection	1)Enter bad area 2)Save	Zero/negative area	Area error	Stays, blocked	Pass
TC _4	Future date validation	Verify future date rejection	1)Enter future date 2)Save	Future work date	Future date error	Stays, blocked	Pass
TC _5	Duplicate work guard	Verify duplicate rejection	1)Save 2)Save again	Same farmer+type +date	Duplicate rejected	Single record kept	Pass
TC _6	Other-work description rule	Verify missing description	1)Select Other 2)Skip desc	Missing description	Description required	Stays, blocked	Pass
TC _7	Work edit lock	Verify direct edit blocked	1)Send PUT directly	PUT edit attempt	405 rejected	Record unchanged	Pass

>>Billing module

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
TC_ 1	Bill generation	Verify bill from unbilled work	1)Select work 2)Generate	Unbilled work	Bill created locked	Unpaid row added	Pass

TC_2	Double billing guard	Verify second bill blocked	1)Select billed 2)Generate	Already -billed work	Already exists error	Single bill kept	Pass
TC_3	Bill total lock	Verify direct edit blocked	1)Send PUT directly	PUT edit attempt	405 rejected	Total unchanged	Pass
TC_4	Bill delete guard	Verify paid bill delete blocked	1)Delete paid bill	Bill having payments	Cannot delete message	Bill + payments kept	Pass
TC_5	Bill status filter	Verify Partial filter	1)Select Partial 2)Apply	Partial status	Filtered bills	Correct rows shown	Pass

>>Payment module

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
TC_1	Payment recording	Verify valid payment	1)Select bill 2)Enter pay 3)Save	Valid amount within pending	Saved, status recalced	Status updated in list	Pass
TC_2	Overpayment guard	Verify excess blocked	1)Enter excess 2)Save	Excess amount	Exceeds error	Stays, blocked	Pass
TC_3	Paid-bill block	Verify Paid bill blocked	1)Select Paid 2)Try pay	Paid bill	Already paid blocked	No payment added	Pass
TC_4	Future date validation	Verify future date	1)Enter future 2)Save	Future payment date	Future error	Stays, blocked	Pass

		blocke d					
TC_ 5	Payment delete + recalc	Verify delete recalcs status	1)Delet e payme nt	Existin g payme nt	Deleted, recalced	Status updated in list	Pass
TC_ 6	Payment edit lock	Verify direct edit blocke d	1)Send PUT directly	PUT edit attempt	405 rejected	Record unchange d	Pass

>>Expense module

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expecte d Result	Post Conditio n	Status (Pass/Fail)
TC_ 1	Expense entry	Verify valid expens e	1)Enter details 2)Save	Valid type + amount + date	Expense saved	Row added in list	Pass
TC_ 2	Zero amount validatio n	Verify zero rejecte d	1)Enter 0 2)Save	Zero amount	Whole- rupee error	Stays, blocked	Pass
TC_ 3	Decimal amount validatio n	Verify decima l rejecte d	1)Enter decima l 2)Save	Decima l amount 100.50	Whole- rupee error	Stays, blocked	Pass
TC_ 4	Future date validatio n	Verify future blocke d	1)Enter future 2)Save	Future expens e date	Future error	Stays, blocked	Pass
TC_ 5	Expense edit lock	Verify direct edit blocke d	1)Send PUT directl y	PUT edit attempt	405 rejected	Record unchange d	Pass

Dashboard + Reports module

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
TC_1	Dashboard totals	Verify dashboard view	1)Login 2)Open Home	—	Totals + recent shown	Cards displayed	Pass
TC_2	Work report filter	Verify filtered report	1)Select Work 2)Apply	Filter values	Filtered report	Correct rows shown	Pass
TC_3	CSV download	Verify CSV export	1)Apply 2)Click CSV	Report data	CSV downloaded	File saved	Pass
TC_4	Pending payment report	Verify pending dues list	1)Select Pending	—	Pending bills listed	Correct dues shown	Pass
TC_5	Language/mode switch	Verify Hindi + Dark mode	1)Change settings	Hindi + Dark mode	UI translated + dark	Display updated	Pass

>>Problem Report module

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
TC_1	Problem report submit	Verify valid report submit	1)Fill form 2)Submit	Valid type + desc + screenshot	Submitted Pending	Row in My Reports	Pass
TC_2	Required-field validation	Verify blank submit blocked	1)Submit blank	Empty form	Required errors	Stays, blocked	Pass

TC_3	Screenshot size rule	Verify oversized blocked	1)Attach 5MB 2)Submit	5 MB file	File rule error	Stays, blocked	Pass
TC_4	Status tracking	Verify status visible	1)Open page	—	Status visible	Pending badge shown	Pass
TC_5	User data isolation	Verify others hidden	1)Search others	Other user's report	Not visible	Own rows only	Pass

>>Administration module

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status (Pass/Fail)
TC_1	Admin dashboard access	Verify superuser access	1)Login admin 2)Open Admin	Admin login	Registry+history+reports	Dashboard displayed	Pass
TC_2	Non-admin block	Verify normal user denied	1)Open /admin URL	Normal user login	403 denied	Stays, blocked	Pass
TC_3	Report status update	Verify admin status change	1)Select report 2)Update	Report + new status	Status updated	Badge changed	Pass
TC_4	Login history view	Verify history listing	1)Open history	—	Date/time/IP rows	Records listed	Pass

6.2 Security Testing

Security Test	Description
Token auth	APIs without token give 401; tokens expire after 24 hours with single active session.

Anti-enumeration	Unknown reset emails get the same generic message; failed logins write no history.
Ownership	Users see only own records; admin APIs need is_superuser (is_staff alone = normal user).
Guards	Overpayment, duplicates, double billing, and linked deletes rejected server-side with 400/405.
Uploads	Screenshots image-type and size validated, served owner-or-admin only with same-404.

6.3 Test Reports

No	Test Name	Executed	Pass	Fail	Total	Bug Tracking
1	Login	6	6	0	6	0
2	Forgot Password	6	6	0	6	0
3	Farmer	6	6	0	6	0
4	Agricultural Work	7	7	0	7	0
5	Billing	5	5	0	5	0
6	Payment	6	6	0	6	0
7	Expense	5	5	0	5	0
8	Dashboard + Reports	5	5	0	5	0
9	Problem Report	5	5	0	5	0
10	Administration	4	4	0	4	0
Total		55	55	0	55	0

All 55 test cases passed with zero failures. Backend automated suite and manual UI testing both confirm the results above.

CHAPTER 7: CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Limitations

- Password reset emails depend on SMTP delivery; the Render free tier to Gmail relay can hang or fail. This is handled with `EMAIL_TIMEOUT=8` and documented for production.
- Free hosting sleeps after periods of inactivity, so the first request after being idle can take up to 50 seconds.
- Only one active session per account is allowed by design; users cannot stay logged in on two devices simultaneously.
- Saved work, bill, payment, and expense records are locked against editing; corrections require deletion and re-entry.
- Problem screenshots are limited to image files up to 2 MB.

7.2 Future Enhancements

- Move transactional email to an HTTP email API (Resend/Brevo) with a verified custom domain for reliable delivery to all users.
- Add a native mobile application that reuses the same REST APIs.
- Add online payment gateway integration with automatic receipt generation.
- Add advanced analytics charts, seasonal income trends, and SMS notifications in Hindi/Marathi.
- Add multi-worker and role-based access for larger agricultural enterprises.

7.3 Conclusion

AgriWorks Manager successfully replaces notebook-based record-keeping with a deployed web application covering farmers, agricultural work with automatic area and irrigation billing, bills with locked totals, payments with pending tracking, expenses, dashboards, reports with CSV export, problem reporting with admin handling, and login

history monitoring. All 55 test cases pass with zero failures. The system is responsive, multilingual, and suitable for small and medium-sized agricultural service businesses, encouraging digital adoption in the agricultural service sector.

REFERENCES

1. React – Official documentation, <https://react.dev/>
2. React Router – Official documentation, <https://reactrouter.com/>
3. Axios – Official documentation, <https://axios-http.com/>
4. Bootstrap – Official documentation, <https://getbootstrap.com/>
5. Vite – Official documentation, <https://vite.dev/>
6. Node.js – Official website and documentation, <https://nodejs.org/>
7. Python – Official documentation, <https://docs.python.org/3/>
8. Django – Official documentation, <https://docs.djangoproject.com/>
9. Django REST Framework – Official documentation, <https://www.django-rest-framework.org/>
10. django-cors-headers – Official repository and documentation, <https://github.com/adamchainz/django-cors-headers>
11. mysqlclient – Official repository and documentation, <https://github.com/PyMySQL/mysqlclient>
12. python-dotenv – Official repository and documentation, <https://github.com/theskumar/python-dotenv>
13. Gunicorn – Official documentation, <https://gunicorn.org/>
14. WhiteNoise – Official documentation, <http://whitenoise.evans.io/>
15. MySQL – Official reference manual, <https://dev.mysql.com/doc/>
16. Vercel – Official documentation, <https://vercel.com/docs>
17. Render – Official documentation, <https://render.com/docs>
18. Aiven – Official documentation, <https://docs.aiven.io/>
19. GitHub – Version control and code hosting, <https://github.com/>
20. OpenCode – AI coding assistant used during development, <https://opencode.ai/>
21. Visual Studio Code – Code editor, <https://code.visualstudio.com/>

CHAPTER 2 REFERENCES

1. S. Fountas et al., Computers and Electronics in Agriculture, vol. 115, pp. 40–50, 2015. <https://doi.org/10.1016/j.compag.2015.05.011>
2. C.G. Sørensen et al., Computers and Electronics in Agriculture, vol. 72, no. 1, pp. 37–47, 2010. <https://doi.org/10.1016/j.compag.2010.02.003>
3. A. Kaloxylas et al., Computers and Electronics in Agriculture, vol. 89, pp. 130–144, 2012. <https://doi.org/10.1016/j.compag.2012.09.002>
4. J. Tummers et al., Precision Agriculture, vol. 22, pp. 22–50, 2021. <https://link.springer.com/article/10.1007/s11119-020-09728-0>
5. Md.S. Basir et al., Precision Agriculture, vol. 25, no. 5, pp. 2643–2682, 2024. <https://link.springer.com/article/10.1007/s11119-024-10172-7>
6. G. Carli et al., Int. J. of Agricultural and Environmental Information Systems, vol. 5, no. 4, pp. 69–84, 2014. <https://www.igi-global.com/article/introducing-activity-based-costing-in-farm-management/120437>
7. M. Cunha et al., Open Agriculture, vol. 4, no. 1, pp. 305–321, 2019. <https://www.degruyter.com/document/doi/10.1515/opag-2019-0029/html>
8. A. Goap et al., Computers and Electronics in Agriculture, vol. 155, pp. 41–49, 2018. <https://www.sciencedirect.com/science/article/abs/pii/S0168169918306987>
9. C. Kamiński et al., Sensors, vol. 19, no. 2, Art. 276, 2019. <https://www.mdpi.com/1424-8220/19/2/276>
10. T. Ojha et al., Computers and Electronics in Agriculture, vol. 118, pp. 66–84, 2015. <https://www.sciencedirect.com/science/article/abs/pii/S0168169915002379>
11. F. J. Ferrández-Pastor et al., Sensors, vol. 16, no. 7, Art. 1141, 2016. <https://www.mdpi.com/1424-8220/16/7/1141>
12. S. Wolfert et al., Agricultural Systems, vol. 153, pp. 69–80, 2017. <https://www.sciencedirect.com/science/article/pii/S0308521X16303754>
13. V. Saiz-Rubio et al., Agronomy, vol. 10, no. 2, Art. 207, 2020. <https://www.mdpi.com/2073-4395/10/2/207>
14. A. Walter et al., Proc. Natl. Acad. Sci. U.S.A., vol. 114, no. 24, pp. 6148–6150, 2017. <https://pnas.org/doi/full/10.1073/pnas.1707462114>

15. D. C. Rose et al., *Frontiers in Sustainable Food Systems*, vol. 2, Art. 87, 2018.
<https://www.frontiersin.org/journals/sustainable-food-systems/articles/10.3389/fsufs.2018.00087/full>